

**POSTS AND TELECOMMUNICATIONS
INSTITUTE OF TECHNOLOGY**

FACULTY OF INFORMATION TECHNOLOGY



REPORT

Analysis and Design of Information Systems

Class: E22HTTT

Project group: 3

Project name: Cinema Management System

Student name: Nguyễn Xuân Kiên

Topic number: 17

Finalized report

A. Requirements

1. Glossary list

No	Name	Meaning
Terms relating to humans		
1	User	Any individual who can login and interact with the cinema management system, must have an account in the system and can only do functions that the system authorizes. This role is the generalization for all specific roles of people who use the system, such as Staff and Customer.
2	Staff	A type of user who is an employee of the cinema. This role is the generalization of Management Staff and Saler.
3	Management staff	A staff role with administrative privileges. They can view the statistics of movies, customers, and revenue. They also have control over the cinema's activities, which includes scheduling showtimes and managing movie and theater information (add, edit, delete).
4	Saler	A staff role focused on in-person, point-of-sale duties at the cinema's counter. Their activities are selling tickets directly to customers and handling the issuance of membership cards.
5	Customer	A user role that interacts with the system for personal use. A customer's activities include registering for membership, searching, buying tickets online, buying tickets at the counter.
Terms relating to activities		
6	Search movie information	The action that a customer performs to look up a movie's details (name, genre, length, description), by entering the name of a movie that they want to find, and the system will display a list of all movies matching the search keyword. The customer can then select a movie and view its details.
7	Viewing statistics	The action that a management staff performs to look up statistical reports of movies, customers or revenue that the system collected. For example the management staff can view the revenues of customers over a specific range of time, and can select a specific customer to view all the transactions that customer made (total price, saleoff, creation time).
8	Schedule showtime	The action that a management staff performs to plan the cinema's screening calendar. This involves assigning specific movies to play in particular theaters at a designated date and time. It also needs to check whether the theater already has a schedule assigned to it on that designated date and time.
9	Manage movie information	The action that a management staff performs to create a new movie, update or delete information about a movie (name, genre,

		length, description)
10	Manage theater information	The action that a management staff performs to create a new theater, update or delete information about a theater (name, type, capacity, description)
11	Selling tickets	The action that a saler performs when a customer purchases tickets at the counter. This involves the saler selecting the ticketing menu asking the customer for the name of the movie and selecting the movie from the list, the saler will then ask the customer for showtime, seats and loyalty card code (if any) and input them into the system, then process the payment and printing out the ticket (movie, cinema, theater, seat, watch time, price, method of order) and the invoice (total price, saleoff, creation time).
12	Issuing membership cards	The action that a saler performs to provide a membership card (card type, saleoff, description, register date) for a customer at the counter, including asking the customer name and phone number and searching for their membership card and handing it out to the customer.
13	Register for membership	The action a customer performs to create a personal membership account by providing their information (username, password, name, date of birth, phone number, email, address) and then clicking register on the interface.
14	Buy ticket	The action a customer performs to purchase tickets, either online through the system's web interface or at the cinema counter with a saler (same as selling tickets activity). For online purchase, the customer selects the ticket purchase menu, then selects a movie from the list, the customer will then select a viewing time, a seat and enter the loyalty card code (if any) for saleoff. Finally the customer confirms the ticket purchase and chooses to pay, the system will print the ticket (movie, cinema, theater, seat, watch time, price, method of order) and invoice (total price, saleoff, creation time).
15	Login	The action a user must perform to interact with the cinema management system. The user can click the log in button, then enter their credentials (username, password) and click Login to log into the website. Depending on the user role, they will be logged into different interfaces (e.g., customer log into customer's interface, management staff log into management staff's interface, saler log into saler's interface).
Terms relating to objects		
16	Movie	An object representing a movie, containing attributes such as name, genre, release date, length, and a description.
17	Theater	An object representing a physical screening room within the cinema, containing attributes such as name, seating capacity and

		description. There are 3 screen types: standard, IMAX, SCREENX, each with their own price
18	Showtime	An object representing a specific time that a movie is shown at a theater, containing the screening time, price attributes. It is the event for which customers can buy tickets.
19	Ticket	An object representing a single admission for a specific showtime. It contains attributes such as movie, cinema, theater, seat, watch time, price, method of order.
20	Membership card	An object representing a card that a customer can own to represent membership in the cinema. It contains attributes such as member name, phone number, card type, description, register date. A membership card is divided into 3 types: bronze, silver and gold. Bronze has 0% saleoff, silver has 15% saleoff, gold has 25% saleoff
21	Seat	An object representing a single, specific place for a customer to sit within a theater. It includes attributes such as name, description. There are 3 types of seats : standard, VIP and VVIP, each with their own price
22	Invoice	An object representing a record of a single purchase made by a customer. It includes attributes such as total price, saleoff, creation time.
23	Statistic	A system-generated summary of data for different objects, such as customer's revenue or movie's revenue, compiled over a specific period of time.
24	Cinema	A public building or venue specifically designed for showing movies, it has attributes such as name, address, description.

2. Project information

2.1. Objective

This system serves the purpose of managing a cinema, which includes operations for different types of users, such as: management staff can view statistics on movies, customers, and revenue, schedule showtimes, and manage information about movies and theaters (including adding, editing, and deleting entries); sales staff can handle ticket sales at the counter and issue membership cards upon request; while customers can register for membership, search for movies, and purchase tickets either online or at the counter.

2.2. Project scope

- User:
 - Login/logout
- Management staff:
 - View statistics

- Schedule showtimes
- Managing movies/theaters information (add, edit, delete)
- Saler:
 - Selling tickets at the counter
 - Issuing membership cards
- Customers:
 - Registering for membership
 - Searching for movies
 - Buying tickets

2.3. Business activities of each modules

- **Customer searches for movie information:** the customer selects the menu to find movie information → the customer enters the movie name to find it → the system displays a list of movies whose name contains the entered keywords → the customer clicks on a movie to view details → the system displays movie details (name, genre, release date, length, description).
- **Management staff views customer statistics:** the management staff selects the menu to view statistical reports → the management staff selects view customer statistics by revenue → the management staff selects start date, end date → the system displays a list of customer sorted by revenue → the management staff clicks on a customer to view details → the system displays detailed statistics of transactions of the selected customer through invoices (number of tickets, total price, saleoff, creation time).

2.4. Information of objects

Group of information relating to human:

- User: username, password, name, date of birth, phone number, email, address, role
- Staff: same as user, but has job role
- Management staff: same as staff
- Saler: same as staff
- Customer: same as user

Group of information relating to cinema operations:

- Membership card: register date
- Card type: name, description, saleoff
- Movie: name, genre, release date, length, description
- Theater: name, seating capacity, description
- Screen type: name, extra price, description
- Seat: name, description
- Seat type: name, extra price, description
- Showtime: screening time, price
- Ticket: price, method of order
- Invoice: total price, saleoff, creation time
- Cinema: name, address, description

Group of information relating to statistics:

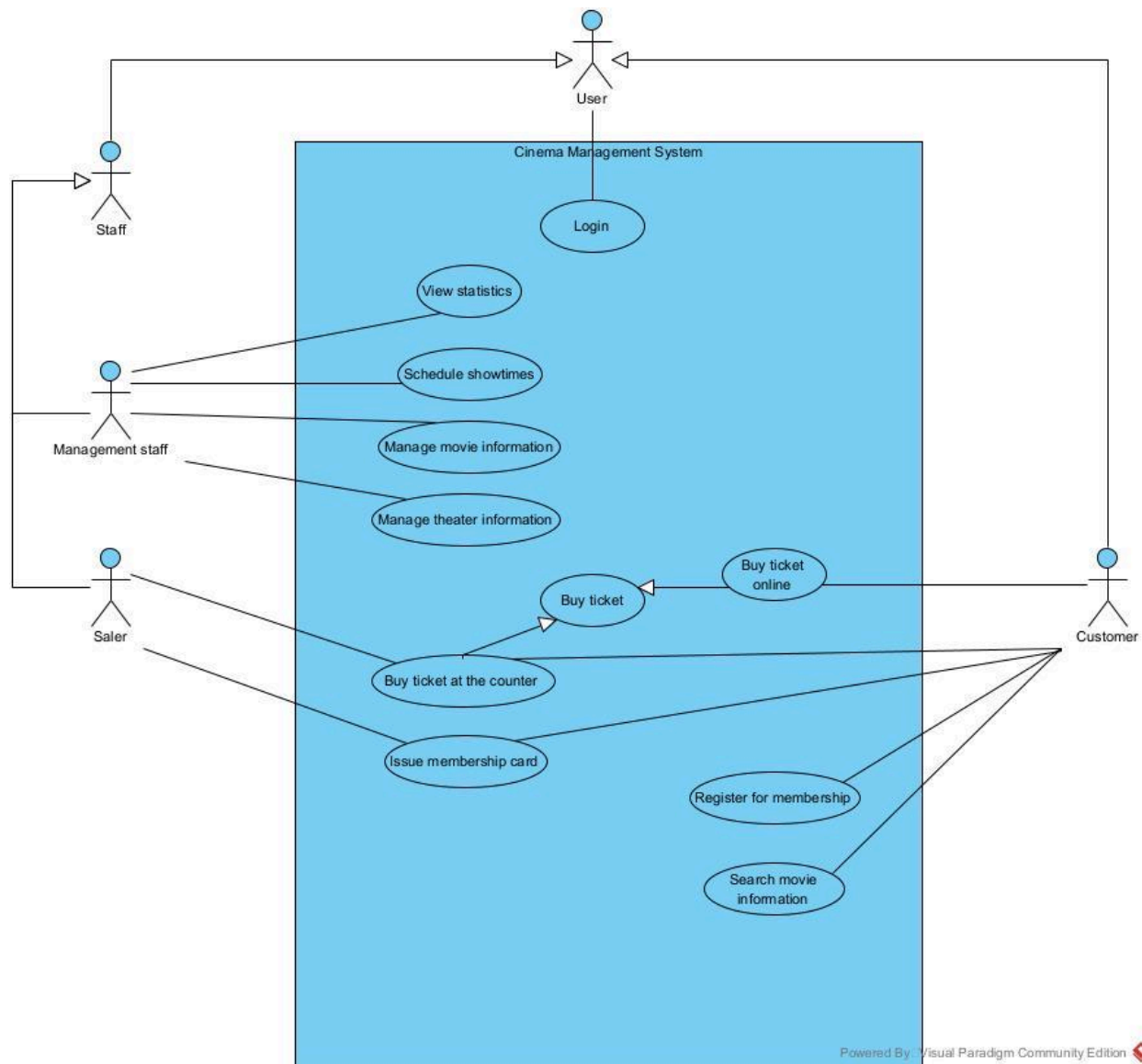
- Customer statistics by revenue
- Movie statistics by revenue

2.5. Relationships among objects

- A customer can only have one membership card
- A membership card can only belong to one customer
- A card type can belong in many membership card
- A membership card can only have one card type
- A customer can have multiple invoices
- An invoice can only belong to one customer
- A ticket can only belong in one invoice
- An invoice can have many tickets
- A ticket can only have one showtime
- A showtime can be in many tickets
- A movie can have many showtimes
- A showtime can only belong to one movie
- A seat can be in many tickets
- A ticket can only have one seat
- A seat type can belong in many seats
- A seat can only have one seat type
- A cinema can have many theaters
- A theater can only belong to one cinema
- A theater can have multiple seats
- A seat can only belong to one theater
- A theater can have many showtimes
- A showtime can only belong to one theater
- A screen type can belong in many theaters
- A theater can only have one screen type
- A saler can be in many invoices
- An invoice can only belong to one saler

3. Use case diagram

3.1. General use case diagram

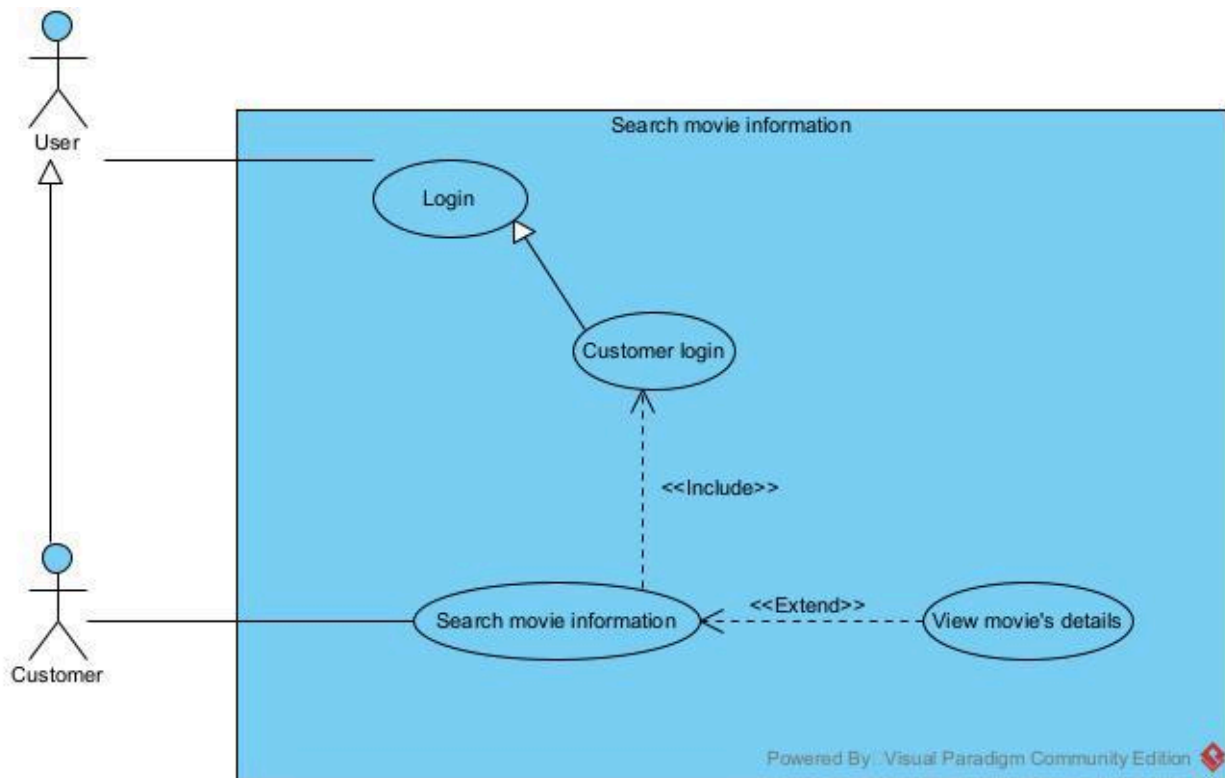


- Description of the use case:

- **Login:** This use case enables the user to log into the cinema management system
- **View statistics:** This use case enables the management staff to view statistics about movies, customers and revenue
- **Schedule showtimes:** This use case enables the management staff to plan the cinema's screening calendar
- **Manage movie information:** This use case enables the management staff to create new movies, change existing movies information or deleting movies
- **Manage theater information:** This use case enables the management staff to create new theaters, change existing theaters information or deleting theaters
- **Search movie information:** This use case enables the customer to be able to search for movies

- Register for membership: This use case enables the customer to be able to register for membership
- Buy ticket online: This use case enables the customer to order tickets on the system interface
- Buy ticket at the counter: This use case enables the customer to buy ticket at the counter with the help of a saler
- Buy ticket online and buy ticket at the counter serve the same purpose => Create use case Buy ticket as generalization
- Issue membership card: This use case enables the saler to issue membership card to the customer

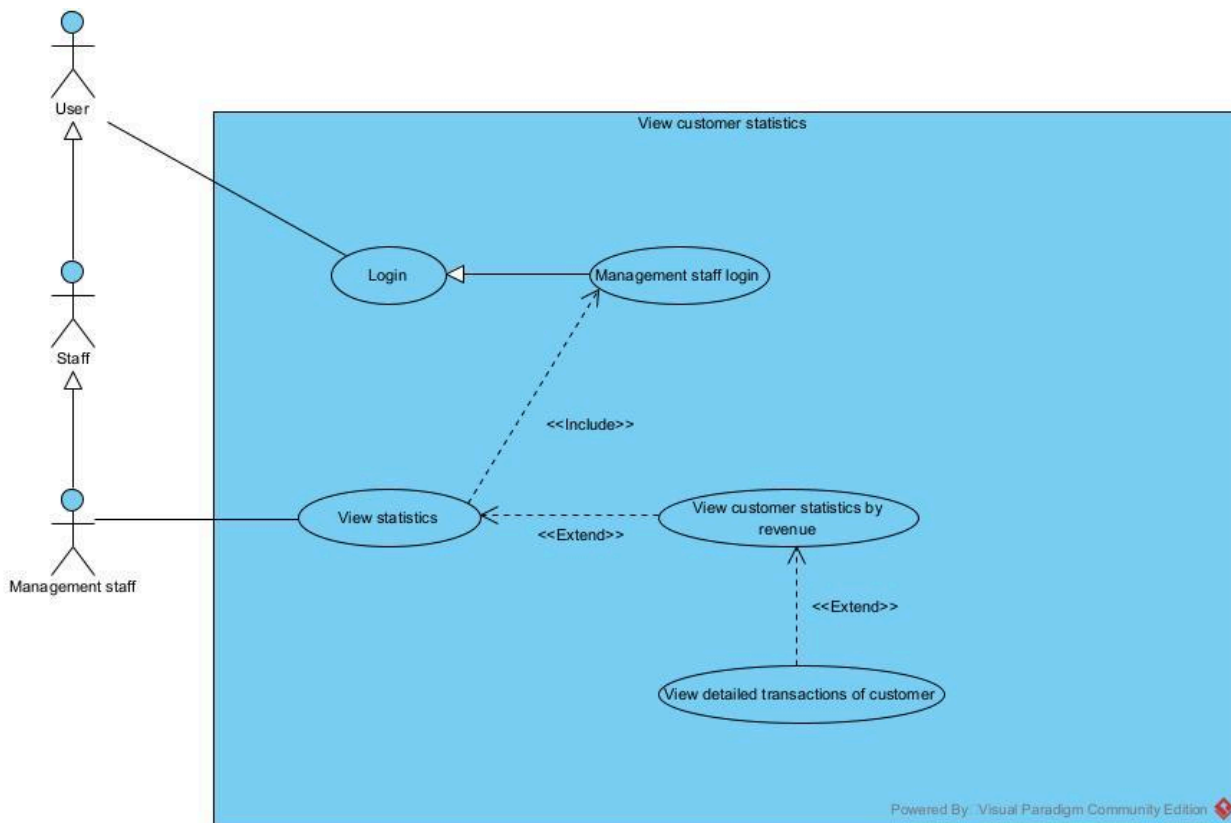
3.2. Use case: *Customer searches for movie information*



- Description of the use case:

- View movie details: This use case enables the customer to see more detailed information of a movie (name, genre, length, description)

3.3. Use case: *Management staff views customer statistics*



- Description of the use case:

- View customer statistics by revenue: This use case enables the management staff to view the main statistical reports of customers sorted by revenue.
- View detailed transactions of customer: this use case enables the management staff to view detailed transactions of a customer through their invoices.

B. Analysis

1. Scenario for each modules

1.1. Customer searches for movie information:

Use case	Search movie information.
Actor	Customer.
Pre-condition	The customer has logged in successfully.
Post-condition	The customer is able to view information relating to the movie they wanted to search.
Main scenario	1. After logging in, the customer chooses the option to search for movies in the main interface. 2. The system displays the search interface, which includes: a search bar, a “Search” button, a table to display search results, a “Back” button and a “Continue” button. 3. The customer enters the key word “Mura” into the search bar and press the search button. 4. The system displays a list of all movie names that contains the word “Mura”:

	Movie name	Release year
	Hát Dưới Mưa	1952
	Cơn Mưa Ngang Qua	2018
	Em Gái Mưa	2018
	Mưa Đỏ	2025
	Giọt Mưa Xuân	2025
	5. The customer press on “Mưa Đỏ” and press on the continue button. 6. This system displays the movie information interface, which has detailed information of the movie, and a back button to return to the search interface: Movie name: Mưa Đỏ Release date: 02/09/2025 Genre: Action Length: 124 minutes Description: A feature movie on the theme of the revolutionary war, scripted by writer Chu Lai, inspired by and fictionalized from the 81 days and nights of heroic, resilient fighting by the people, cadres, and soldiers defending Quảng Trị Citadel in 1972.	
Exception(s)	4. The system does not find any movies matching the keyword “Mưa”.	

1.2. Management staff views customer statistics

Use case	View customer statistics by revenue.
Actor	Management staff.
Pre-condition	The management staff has logged in successfully.
Post-condition	- The management staff is able to view a list of statistics of customers sorted by revenue in a time period. - The management staff is able to view the detailed transactions of a customer through their invoices by clicking on a specific customer in the statistics list.
Main scenario	1. After logging in, the management staff chooses the option to view statistics in the main interface. 2. The system displays the view statistics interface, which has an option to view statistics of customers by revenue. 3. The management staff presses the button to view statistics of customers by revenue. 4. The system displays the view customer statistics by revenue interface, which has: an input date for start date, an input date for end date, a “Search” button, a table for displaying customer revenue statistics, a “Next” button and a “Back” button. 5. The management staff enters the start date as “08/06/2025”, and enters the end date as “18/06/2025” and presses the “Search” button.

6. The system displays a list of customers, sorted from highest to lowest revenue during the time period:

ID	Customer name	Tickets purchased	Total revenue (in VND)
11	Pham Minh Hoang	5	11400000
5	Tran Thanh Y	3	420000
4	Nguyen Van X	3	357000
3	Dang Thu C	2	221000
7	Nguyen Thi B	2	210000
2	Nguyen Van A	2	180000
6	Nguyen Xuan A	0	0
12	Vu Ngoc Anh	0	0

7. The management staff selects Pham Minh Hoang and presses the “Next” button.

8. The system displays the detailed transactions interface of customer Pham Minh Hoang, which has some information about the customer, a list which has all the transactions that Pham Minh Hoang made in the time period, a “Back” button and a “Home” button:

Customer code: 11

Customer name: Pham Minh Hoang

Customer date of birth: 20/01/2001

Customer phone: 0912222222

Customer email: hoangpm@example.com

Customer address: 15 Le Loi, Hai Phong

ID	Movie	Cinema	Theater	Seats	Watch time	Total price (in VND)	Saleoff	Method of order	Creation time	Saler
4	The Dark Knight	Galaxy Cinema	Theater 1	B5, B6	2025-06-15 20:15:00	300000		Counter	2025-06-15 20:00:00	Phung Canh Mo
7	Mua Đò	Lotte Cinema Da Nang	Theater 4	E1, E2, E3	2025-06-19 13:30:00	840000		Online	2025-06-18 19:00:00	Phung Canh Mo

Exception(s)

6. There are no customers

2. Class extraction of the system

2.1. System description

This system serves the purpose of managing a cinema, which includes operations for different types of users such as staff and customers. Management staff can view statistics on movies, customers based on revenue (can also view tickets count), schedule showtimes, and manage information about movies and theaters (including adding, editing, and deleting entries). Sales staff can handle selling tickets at the counter for customers, print out invoices for transactions; and issue membership cards upon request. Customers can register for membership cards (which have different types), search for movies and view movie information which are name, genre, release date, length, description; and purchase tickets either online or at the counter, in which they can choose movies, theaters (which have different screen types), showtime, seats (which have different seat types).

2.2. Noun extraction

- Nouns relating to humans: Management staff, saler, customer, user, staff
- Nouns relating to objects: Cinema, theaters, tickets, invoices, membership cards, seats, system, operations, counter
- Nouns relating to information: Movie statistics, customer statistics, showtimes, movies, name, genre, release date, length, description, revenue, ticket count, information, entries, transactions, screen type, seat type, membership card type

2.3. Classify all the nouns

- Nouns: system, operations, counter, information, entries, transactions => Too abstract, reject
- Nouns relating to humans:
 - User => Class User (abstract): username, password, name, date of birth, phone number, email, address, role
 - Staff => Class Staff (abstract): inherited from User, job role
 - Management staff => Class ManagementStaff: inherited from Staff
 - Saler => Class Saler: inherited from Staff
 - Customer => Class Customer: inherited from User
- Nouns relating to objects:
 - Cinema => Class Cinema: name, address, description
 - Theater => Class Theater: name, seating capacity, description
 - Ticket => Class Ticket: price, method of order
 - Invoice => Class Invoice: total price, saleoff, creation time
 - Membership card => Class MembershipCard: card type, description, register date
 - Seat => Class Seat: name, description
- Nouns relating to information:
 - Movie => Class Movie: name, release date, genre, length, description
 - Showtime => Class Showtime: screening time, price
 - Movie statistics => Class MovieStats: revenue, ticket count
 - Customer statistics => Class CustomerStats: revenue, ticket count
 - Name, genre, release date, length, description => Attributes for class Movie
 - Screen type => Class ScreenType: name, extra price, description

- Seat type => Class SeatType: name, extra price, description
- Membership card type => Class CardType: name, description, saleoff
- Revenue, ticket count => Attribute for Classes MovieStats and CustomerStats

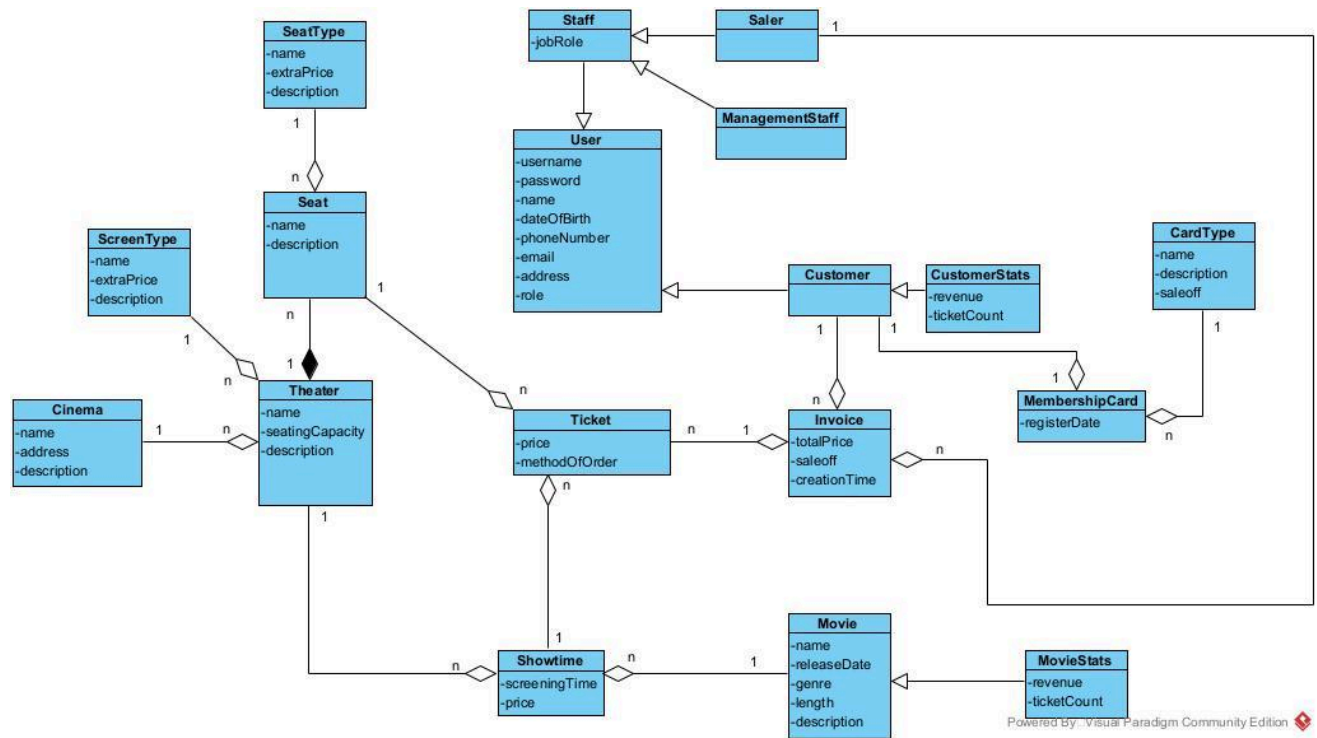
2.4. Consider the quantitative relationship between entity classes

- A cinema can have many theaters => Relationship between Cinema and Theater is 1-n
- A screen type can be in many theaters => Relationship between ScreenType and Theater is 1-n
- A theater can have many seats => Relationship between Theater and Seat is 1-n
- A theater can have many showtimes => Relationship between Theater and Showtime is 1-n
- An invoice can have many tickets => Relationship between Invoice and Ticket is 1-n
- A movie can have many showtimes => Relationship between Movie and Showtime is 1-n
- A seat type can be in many seats => Relationship between SeatType and Seat is 1-n
- A seat can be in many tickets => Relationship between Seat and Ticket is 1-n
- A showtime can have many tickets => Relationship between Showtime and Ticket is 1-n
- A customer can have many invoices => Relationship between Customer and Invoice is 1-n
- A customer can only have one membership card => Relationship between MembershipCard and Customer is 1-1
- A card type can be in many memberships cards => Relationship between CardType and MembershipCard is 1-n
- A saler can be in many invoices => Relationship between Saler and Invoice is 1-n

2.5. Consider the object relationship between entity classes

- Cinema is a component of Theater
- ScreenType is a component of Theater
- Seat is component of Theater
- Seat is a component of Ticket
- SeatType is a component of Seat
- Ticket is component of Invoice
- Movie is component of Showtime
- Theater is component of Showtime
- Showtime is a component of Ticket
- Saler is a component of Invoice
- Customer is a component of Invoice
- Customer is a component of MembershipCard
- CardType is a component of MembershipCard
- Customer inherits from User
- Staff inherits from User
- ManagementStaff inherits from Staff
- Saler inherits from Staff
- CustomerStats inherits from Customer
- MovieStats inherits from Movie

We have the class diagram of the system:



3. Full class diagram of each modules

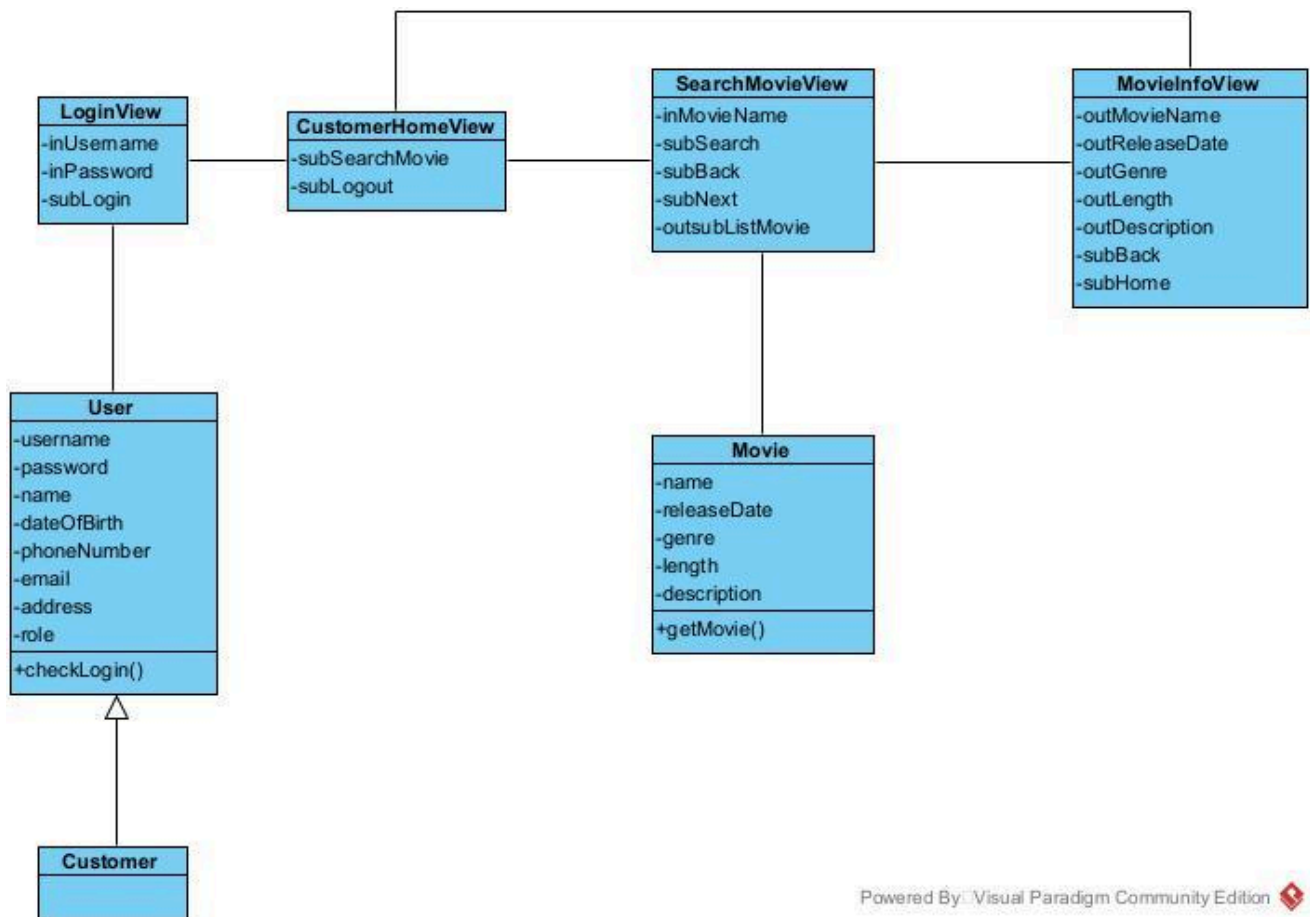
3.1. Customer searches for movie information

a. Analyzing the module

- The customer enters the system => The login interface appears => Needs a class: LoginView, which has:
 - Input for username => inUsername
 - Input for password => inPassword
 - A button to login => subLogin
- The customer enters the username and password => The system must check if the username and password is correct => Needs a method:
 - Name: checkLogin()
 - Input: username, password (of class User)
 - Output: boolean
 - Assign to entity class: User
- Once login is successful => The customer main interface appears => Needs a class: CustomerHomeView, which has at least:
 - An option to search for movie information => subSearchMovie
 - A button to logout => subLogout
- The customer chooses the option to search for movie => The search movie interface appears => Needs a class: SearchMovieView, which has:
 - A search bar to search for movie name => inMovieName
 - A button to search for movie => subSearch
 - A button to return to the previous interface => subBack
 - A button to view the detailed information of a movie => subNext
 - A list of all movies whose name contains the search keyword => outsubListMovie

- The customer enters the keyword into the search bar => The system has to query all the movie whose name contain that keyword => Needs a method:
 - Name: getMovie()
 - Input: Keyword
 - Output: List of Movie
 - Assign to entity class: Movie
- The results are displayed on SearchMovieView
- The customer clicks on a movie to view its detailed information => The interface for detailed movie information appears => Needs a class: MovieInfoView, which has:
 - Movie name => outMovieName
 - Release date => outReleaseDate
 - Genre => outGenre
 - Length => outLength
 - Description => outDescription
 - A button to return to the previous interface => subBack
 - A button to return to the customer home interface => subHome
- We have already queried all the movie in SearchMovieView => No need for a method to query a single movie, we can use the list of movie in SearchMovieView to find a particular movie

b. Full class diagram



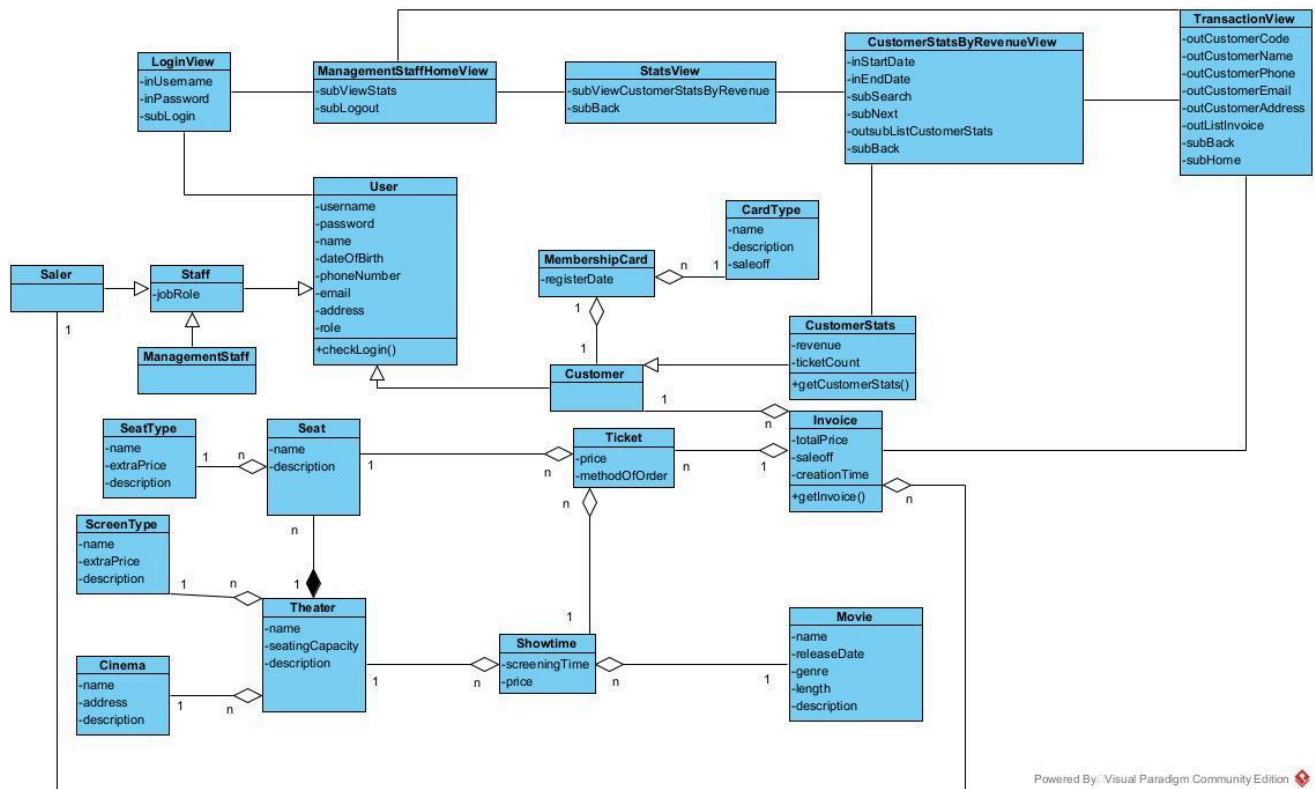
3.2. Management staff views customer statistics

a. Analyzing the module

- The management staff enters the system => The login interface appears => Needs a class: LoginView, which has:
 - Input for username => inUsername
 - Input for password => inPassword
 - A button to login => subLogin
- The management staff enters the username and password => The system must check if the username and password is correct => Needs a method:
 - Name: checkLogin()
 - Input: username, password (of class User)
 - Output: boolean
 - Assign to entity class: User
- Once login is successful => The management staff main interface appears => Needs a class: ManagementStaffHomeView, which has at least:
 - An option to choose to view statistics: subViewStats
 - A button to logout: subLogout
- The management staff chooses to view statistics => The statistics interface appears => Needs a class: StatsView, which has at least:
 - An option to choose to view customer statistics by revenue => subViewCustomerStatsByRevenue
 - A button to return to the previous interface => subBack
- The management staff chooses the option to view customer statistics by revenue => The customer statistics interface appears => Needs a class: CustomerStatsByRevenueView, which has:
 - An input start date => inStartDate
 - An input end date => inEndDate
 - A button to search: subSearch
 - A button to go to view transactions of chosen customer: subNext
 - A list of all customer with their statistics => outsubListCustomerStats
 - A button to return to the previous interface => subBack
- The management staff enters the desired start date, end date and then clicks on the “Search” button => The system has to query all the customer statistics => Needs a method:
 - Name: getCustomerStats()
 - Input: Start date and end date
 - Output: List of CustomerStats
 - Assign to entity class: CustomerStats
- The results are displayed on CustomerStatsByRevenueView
- The management staff clicks on a customer to view their transactions in the time period specified => The interface of detailed transactions of customer appears => Needs a class: TransactionView, which has:
 - Customer code => outCustomerCode
 - Customer name => outCustomerName
 - Customer phone => outCustomerPhone

- Customer email => outCustomerEmail
- Customer address => outCustomerAddress
- A list which has all the invoices of that customer in the time period => outListInvoice
- A button to return to the previous interface => subBack
- A button to return to the management staff home interface => subHome
- The system needs to query all the invoices relating to the customer in the specified time period => Needs a method:
 - Name: getInvoice()
 - Input: Start date, end date and customer code
 - Output: List of Invoice
 - Assign to entity class: Invoice

b. Full class diagram



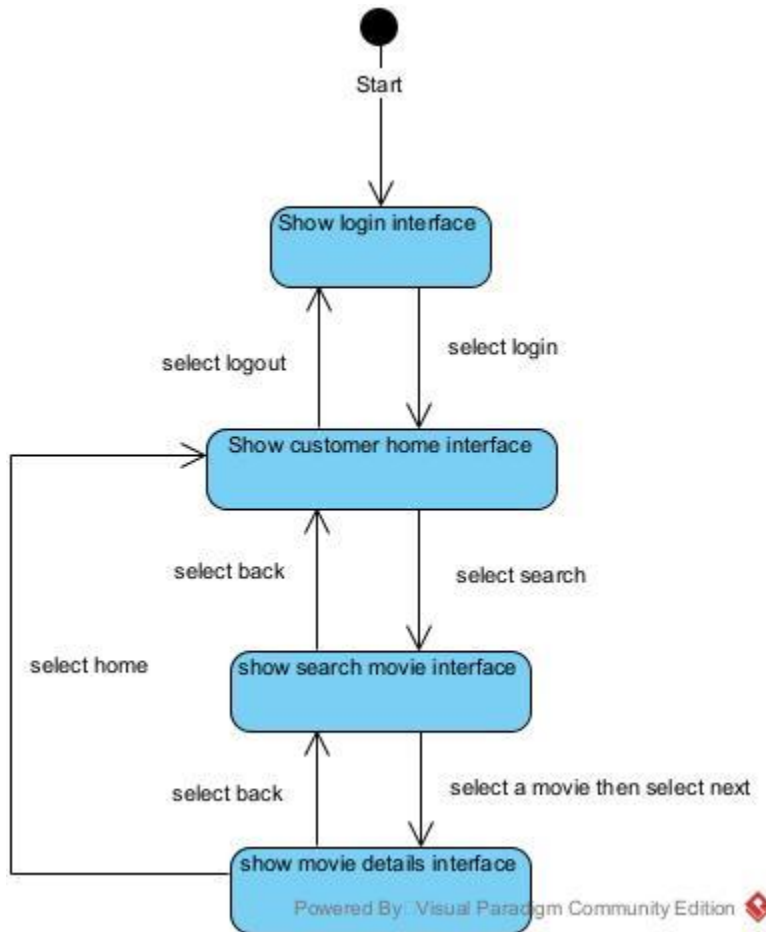
4. Activity analysis of each modules

4.1. Customer searches for movie information

a. State diagram

1. From the login interface, after entering username and password and pressing “Login”, the system shows the customer home interface
2. From the customer home interface, after selecting the option to search for movie, the system shows the search movie interface
3. From the search movie interface, after entering the keyword on the search bar and press “Search”, then selecting a movie from the list of movies whose name contain the keyword and press “Next”, the system shows the movie details interface

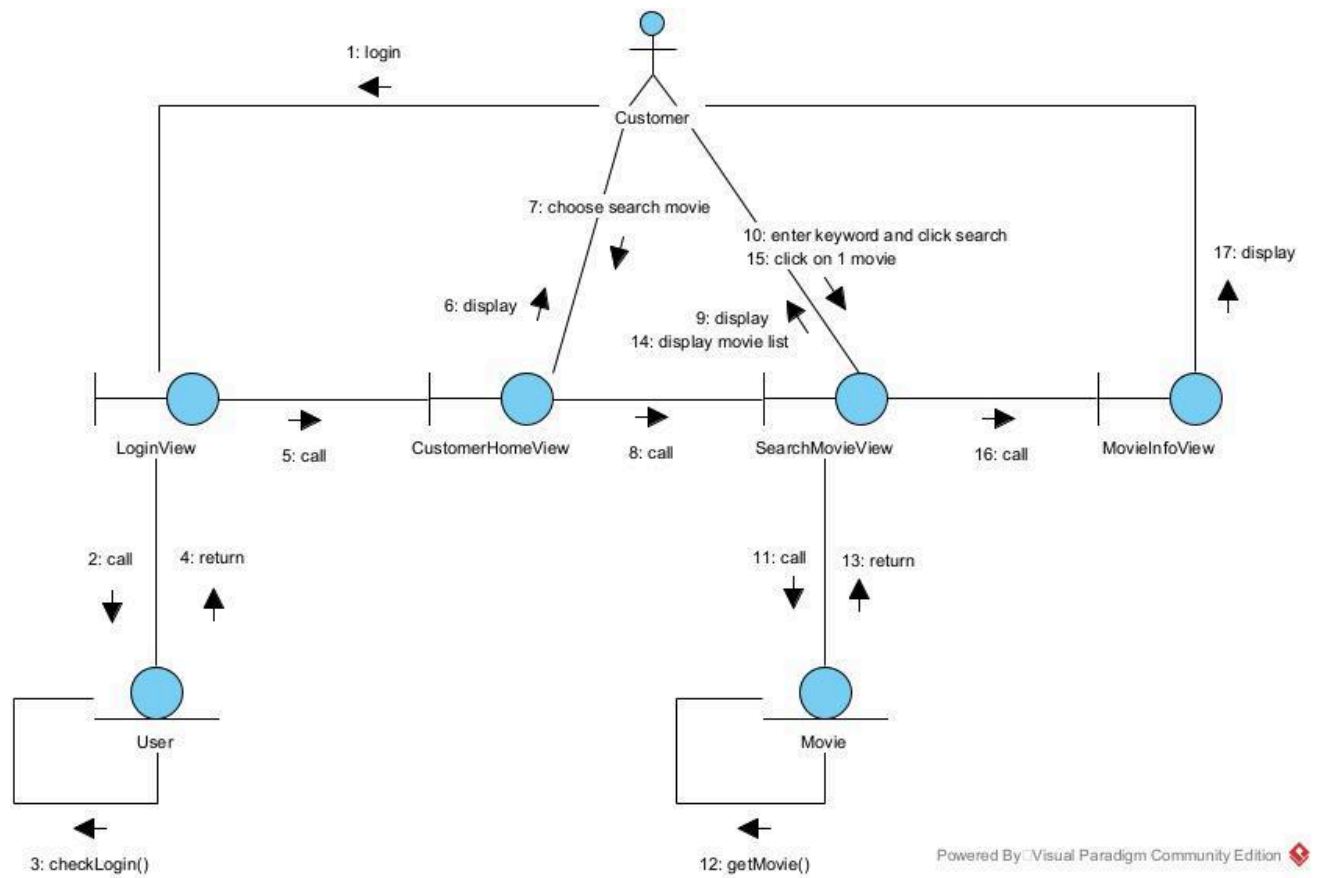
4. In the movie details interface, the customer can choose “Home” button to return to the customer home interface
5. In all the interface, choosing “Back” (“Logout” for customer home interface) will return to the previous interface



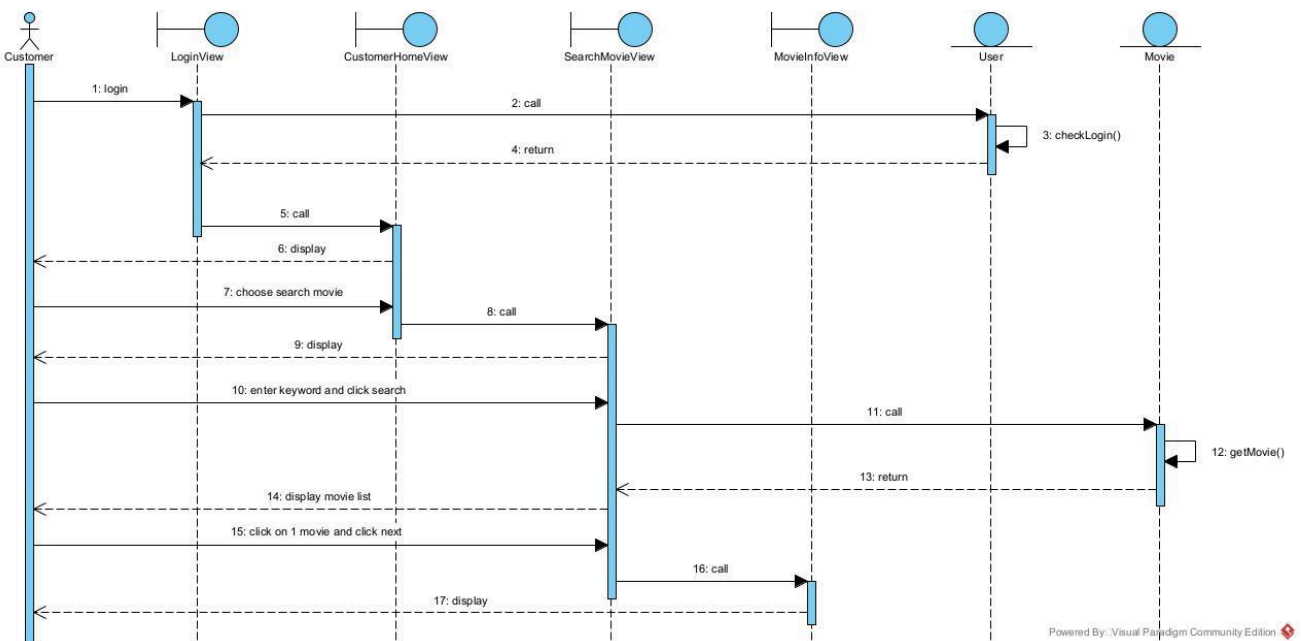
b. Scenario version 2

1. The customer clicks the “Login” button on the interface after entering username and password
2. The class LoginView calls the class User
3. The class User calls the method checkLogin()
4. The class User returns result to the class LoginView
5. The class LoginView calls the class CustomerHomeView
6. The class CustomerHomeView displays to the customer
7. The customer chooses the option to search for movie
8. The class CustomerHomeView calls the class SearchMovieView
9. The class SearchMovieView displays to the customer
10. The customer enters the keyword into the search bar and clicks “Search”
11. The class SearchMovieView calls the class Movie
12. The class Movie calls the method getMovie()
13. The Movie return results to the class SearchMovieView
14. The class SearchMovieView displays the result to the customer
15. The customer clicks on a movie and clicks on the “Next” button to view its details
16. The class SearchMovieView calls the class MovieInfoView
17. The class MovieInfoView displays to the customer

c. Communication diagram



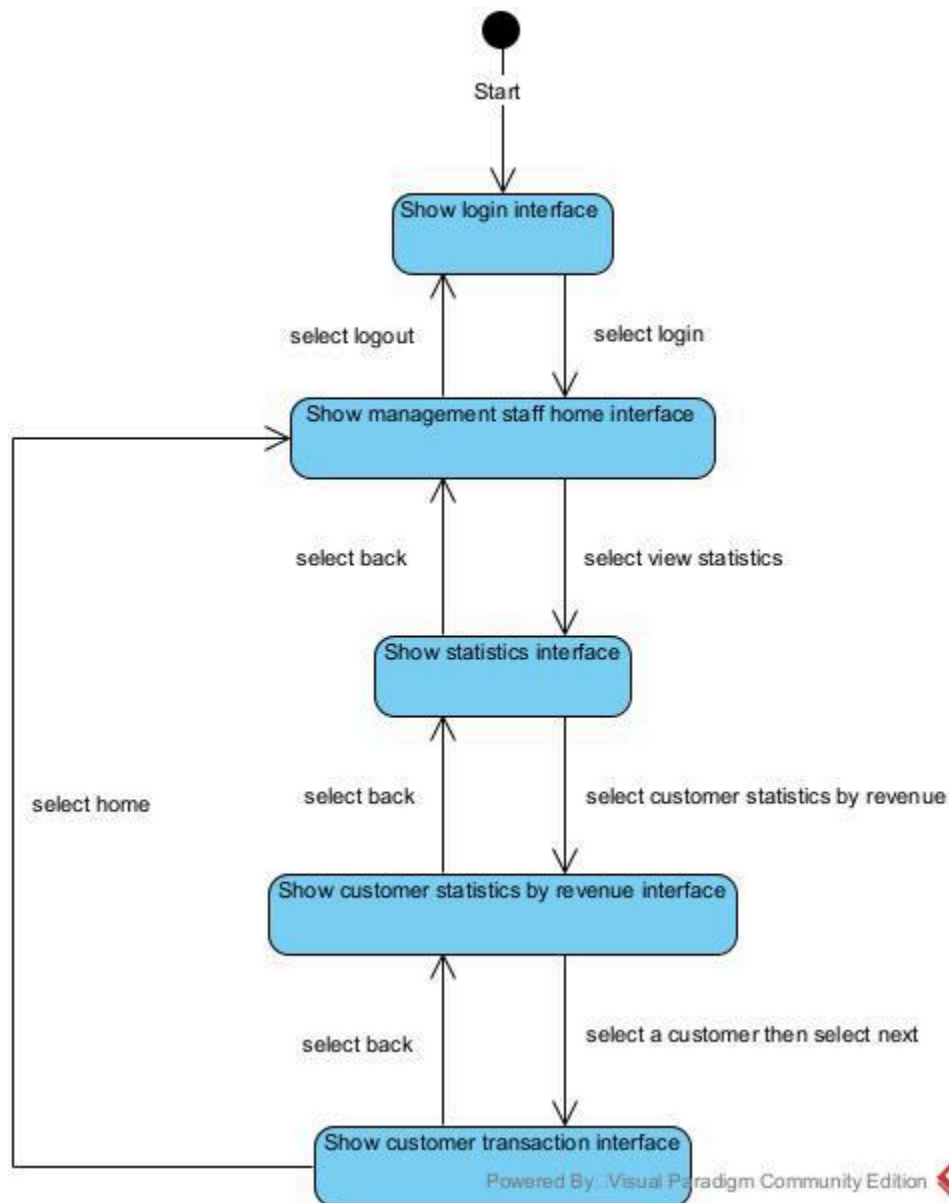
d. Sequence diagram



4.2. Management staff views customer statistics

a. State diagram

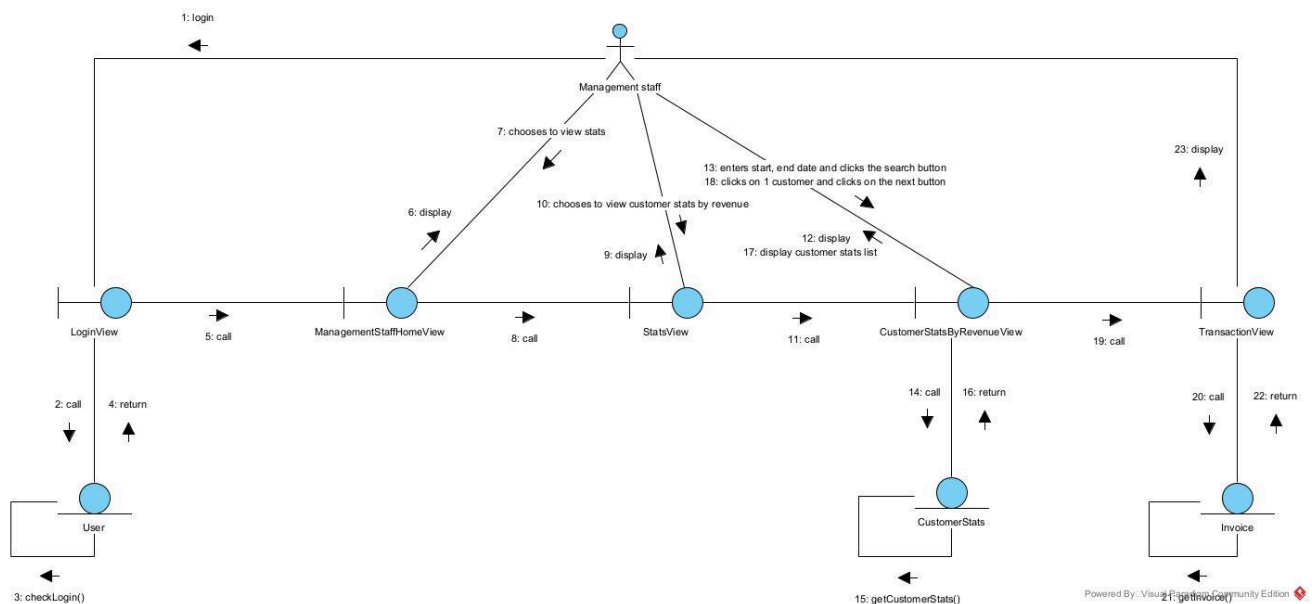
1. From the login interface, after entering username and password and pressing “Login”, the system shows the management staff home interface
2. From the management staff home interface, after selecting the option to view statistics, the system shows the statistics interface
3. From the statistics interface, after selecting the option to view customer statistics by revenue, the system shows the customer statistics by revenue interface
4. From the customer statistics by revenue interface, after entering the start date and end date and clicking the “Search” button, then clicking on a customer and clicking on the “Next” button, the system shows the customer transaction interface
5. In the customer transaction interface, the management staff can choose “Home” button to return to the management staff home interface
6. In all the interface, choosing “Back” (“Logout” for management staff home interface) will return to the previous interface



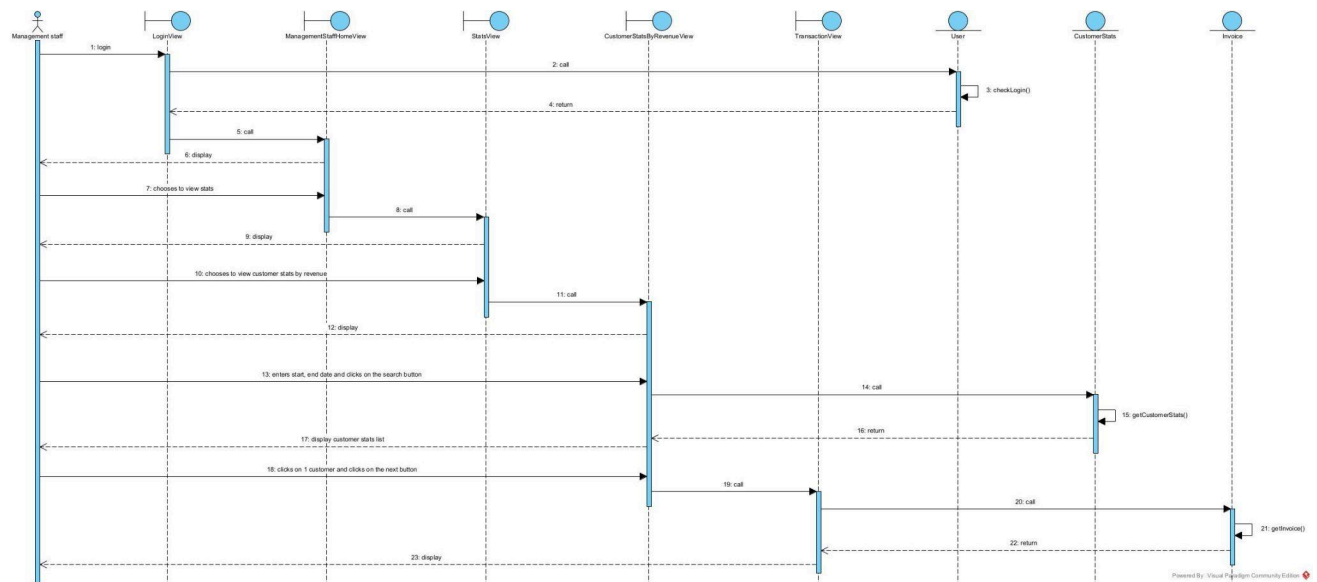
b. Scenario version 2

1. The management staff clicks the “Login” button on the interface after entering username and password
2. The class LoginView calls the class User
3. The class User calls the method checkLogin()
4. The class User returns result to the class LoginView
5. The class LoginView calls the class ManagementStaffHomeView
6. The class ManagementStaffHomeView displays to the management staff
7. The management staff chooses the option to view statistics
8. The class ManagementStaffHomeView calls the class StatsView
9. The class StatsView displays to the customer
10. The management staff chooses the option to view statistics of customers by revenue
11. The class StatsView calls the class CustomerStatsByRevenueView
12. The class CustomerStatsByRevenueView displays to the management staff
13. The management staff enters the start date, end date and then clicks on the “Search” button
14. The class CustomerStatsByRevenueView calls the class CustomerStats
15. The class CustomerStats calls the method getCustomerStats()
16. The class CustomerStats returns the result to the class CustomerStatsByRevenueView
17. The class CustomerStatsByRevenueView displays the result to the management staff
18. The management staff clicks on a customer and clicks on the “Next” button to view the transactions of the customer
19. The class CustomerStatsByRevenueView calls the class TransactionView
20. The class TransactionView calls the class Invoice to get data
21. The class Invoice calls the method getInvoice()
22. The class Invoice returns the result to the class TransactionView
23. The class TransactionView display the result to the management staff

c. Communication diagram



d. Sequence diagram



C. Design

1. System class diagram of the design phase

- Step 1: Add the id attribute for the classes which do not inherit from other class: User, Cinema, Theater, Seat, Showtime, Movie, Ticket, Invoice, MembershipCard, SeatType, CardType
- Step 2: Add the type of each attribute in each classes
- Step 3: Convert all association relationships to correspond aggregation/composition relationships:
 - There are no association relationships => No need
- Step 4: Add the object attribute which correspond to the aggregation/composition relationships:
 - Customer is a component of Invoice, of type 1 - n -> Invoice has Customer
 - Customer is a component of MembershipCard, of type 1 - 1 -> MembershipCard has Customer
 - CardType is a component of MembershipCard, of type 1-n -> MembershipCard has CardType
 - Cinema is a component of Theater, of type 1 - n -> Theater has Cinema
 - SeatType is a component of Seat, of type 1-n -> Seat has SeatType
 - Seat is a component of Theater, of type n - 1 -> Theater has a list of Seat
 - Seat is a component of Ticket, of type 1 - n -> Ticket has Seat
 - Showtime is a component of Ticket, of type 1 - n -> Ticket has Showtime
 - Movie is a component of Showtime, of type 1 - n -> Showtime has Movie
 - Theater is a component of Showtime, of type 1 - n -> Showtime has Theater
 - ScreenType is a component of Theater, of type 1-n -> Theater has ScreenType
 - Ticket is a component of Invoice, of type n - 1 -> Invoice has a list of Ticket
 - Saler is a component of Invoice, of type 1 - 1 -> Invoice has Saler

- tblMembershipCard: id, registerDate
- tblCardType: id, name, saleoff, description
- tblInvoice: id, totalPrice, saleoff, creationTime
- tblTicket: id, price, methodOfOrder
- tblCinema: id, name, address, description
- tblTheater: id, name, seatingCapacity, description
- tblScreenType: id, name, extraPrice, description
- tblSeat: id, name, description
- tblSeatType: id, name, extraPrice, description
- tblShowtime: id, screeningTime, price
- tblMovie: id, name, releaseDate, genre, length, description
- tblMovieStats: revenue, ticketCount
- tblCustomerStats: revenue, ticketCount

Step 3: Consider the quantity relationships among entity classes:

- 1 tblStaff - 1 tblUser (tblStaff inherits from tblUser)
- 1 tblSaler - 1 tblStaff (tblSaler inherits from tblStaff)
- 1 tblManagementStaff - 1 tblStaff (tblManagementStaff inherits from tblStaff)
- 1 tblCustomer - 1 tblUser (tblCustomer inherits from tblUser)
- 1 tblCustomer - 1 tblCustomerStats (tblCustomerStats inherits from tblCustomer)
- 1 tblSaler - 0..n tblInvoice
- 1 tblCustomer - 0..n tblInvoice
- 1 tblCustomer - 0..1 tblMembershipCard
- 1 tblCardType - 1...n tblMembershipCard
- 1 tblCinema - 1..n tblTheater
- 1 tblTheater - 1..n tblSeat
- 1 tblScreenType - 1..n tblTheater
- 1 tblSeatType - 1..n tblSeat
- 1 tblShowtime - 1..n tblTicket
- 1 tblMovie - 0..n tblShowtime
- 1 tblMovie - 1 tblMovieStats (tblMovieStats inherits from tblMovie)
- 1 tblSeat - 1..n tblTicket
- 1 tblInvoice - 1..n tblTicket

Step 4: Config the key columns for tables:

- tblStaff inherits from tblUser -> tblStaff has foreign key tblUserid
- tblSaler inherits from tblStaff -> tblSaler has foreign key tblStafftblUserid
- tblManagementStaff inherits from tblStaff -> tblManagementStaff has foreign key tblStafftblUserid
- tblCustomer inherits from tblUser -> tblCustomer has foreign key tblUserid
- tblCustomerStats inherits from tblCustomer -> tblCustomerStats has foreign key tblUserid
- 1 tblSaler - 0..n tblInvoice -> tblInvoice has foreign key tblSalertblUserid
- 1 tblCustomer - 0..n tblInvoice -> tblInvoice has foreign key tblCustomertblUserid
- 1 tblCustomer - 0..1 tblMembershipCard -> tblMembershipCard has foreign key tblUserid
- 1 tblCardType - 1...n tblMembershipCard -> tblMembershipCard has foreign key tblCardTypeid

3. Detailed modules design

3.1. Static design

a. Customer searches for movie information

Interface design:

LoginView.jsp

Cinema System

Login

Username:

Password:

CustomerHomeView.jsp

Cinema System

Customer Home

Nguyen Van A

Cinema System

Movie Search

Please enter the
keyword:

Mưa

Search

Movie name	Release year
Mưa Đỏ	2025
Em Gái Mưa	2018
Con Mưa Ngang Qua	2018
Hát Dưới Mưa	1952

Back

Next

Cinema System

Movie Information

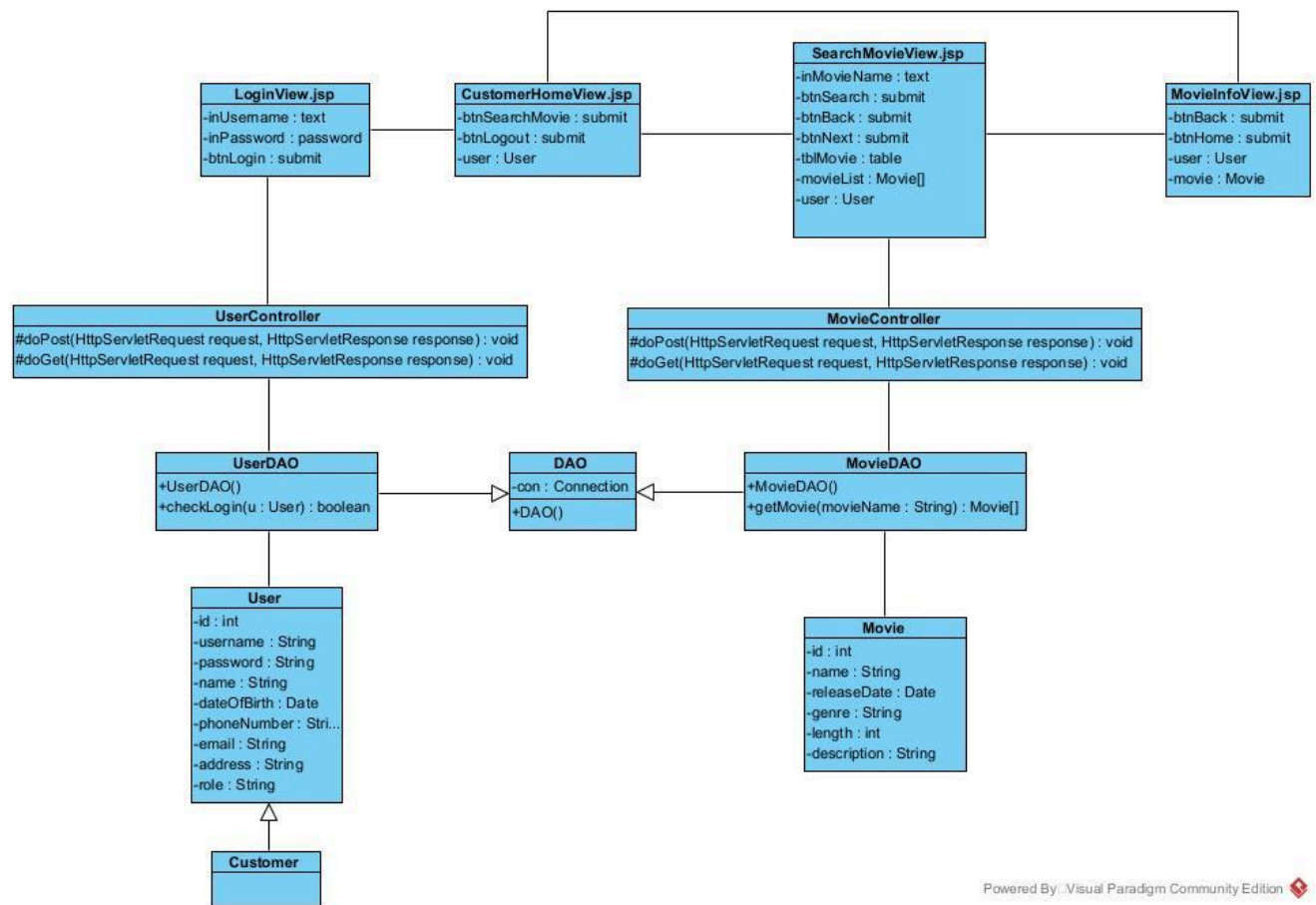
Name: Mưa Đỏ
Release date: 22/08/2025
Genre: Action
Length: 124 minutes
Description:

A feature movie on the theme of the revolutionary war, scripted by writer Chu Lai, inspired by and fictionalized from the 81 days and nights of heroic, resilient fighting by the people, cadres, and soldiers defending Quảng Trị Citadel in 1972.

[Back](#)[Home](#)

Class diagram:

- Interface layer has the following jsp pages: LoginView.jsp, CustomerHomeView.jsp, SearchMovieView.jsp, MovieInfoView.jsp
- Controller layer has the following Controller classes: UserController, MovieController
- Data access layer has the following DAO classes: UserDAO, MovieDAO, DAO



Powered By: Visual Paradigm Community Edition

b. Management staff views customer statistics

Interface design:

LoginView.jsp

Cinema System

Login

Username:

Password:

Cinema System

Management Staff Home

Nguyen Xuan Kien

View Statistics

Logout

Cinema System

Statistic

Customer Statistics

Back

Cinema System

Customer Statistics by Revenue

From: 08/06/2025

To: 18/06/2025

View

Customer Code	Customer name	Tickets purchased	Total revenue (in VND)
11	Pham Minh Hoang	5	1140000
4	Nguyen Van X	3	420000
5	Tran Thanh Y	3	420000
3	Dang Thu C	2	221000
7	Nguyen Thi B	2	210000
2	Nguyen Van A	2	180000
6	Nguyen Xuan A	0	0
12	Vu Ngoc Anh	0	0

Total tickets purchased: 17

Total revenue: 2591000 VND

Back

Next

Cinema System

Customer Transactions

Customer code: 11

Customer name: Pham Minh Hoang

Customer date of birth: 2001-01-20

Customer phone: 0912222222

Customer email: hoangpm@example.com

Customer address: 15 Le Loi, Hai Phong

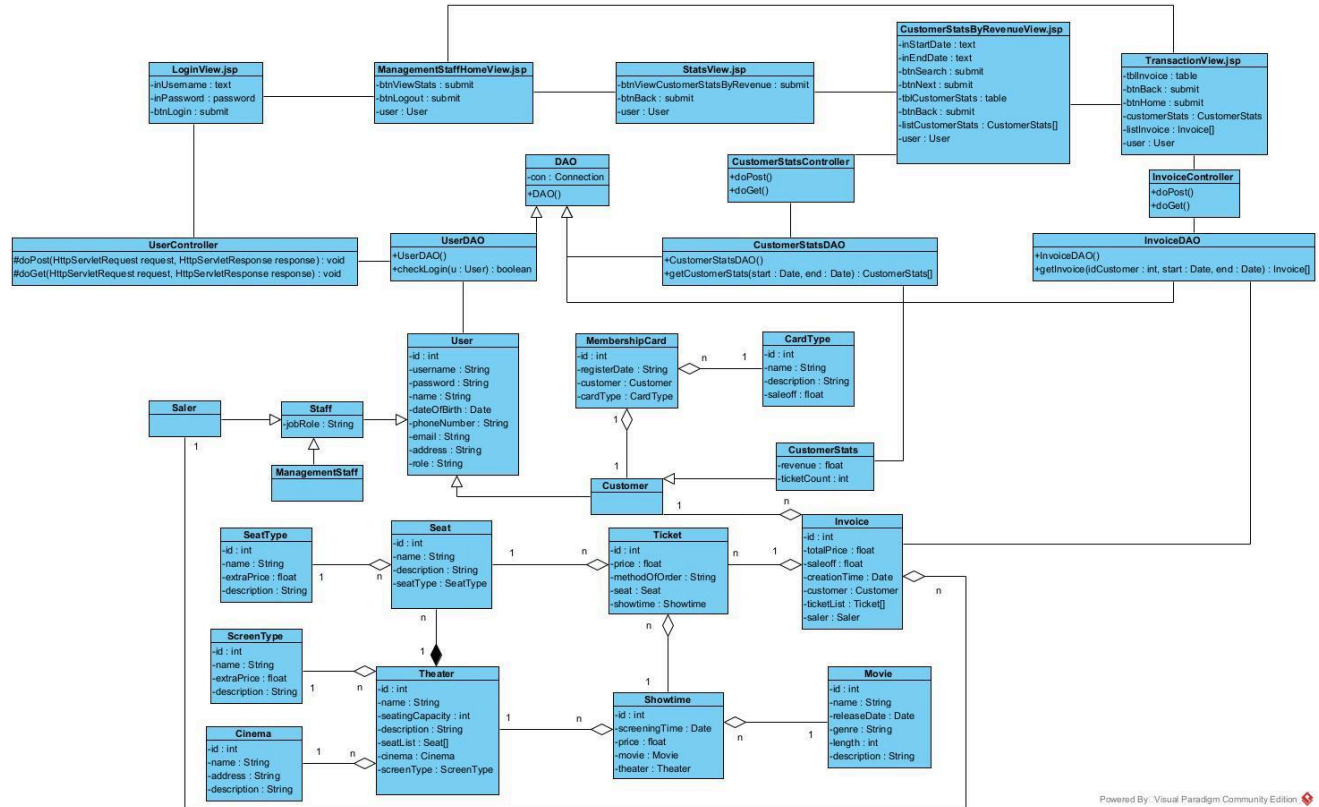
ID	Movie	Cinema	Theater	Seats	Watch time	Total price (in VND)	Saleoff	Method of order	Creation time	Saler
4	The Dark Knight	Galaxy Cinema	Theater 1	B5, B6	2025-06-15 20:15:00	300000	0%	counter	2025-06-15 20:00:00	Phung Canh Mo
7	Mưa Đỏ	Lotte Cinema Da Nang	Theater 4	E1, E2, E3	2025-06-19 13:30:00	840000	0%	online	2025-06-18 19:00:00	Phung Canh Mo

Back

Home

Class diagram:

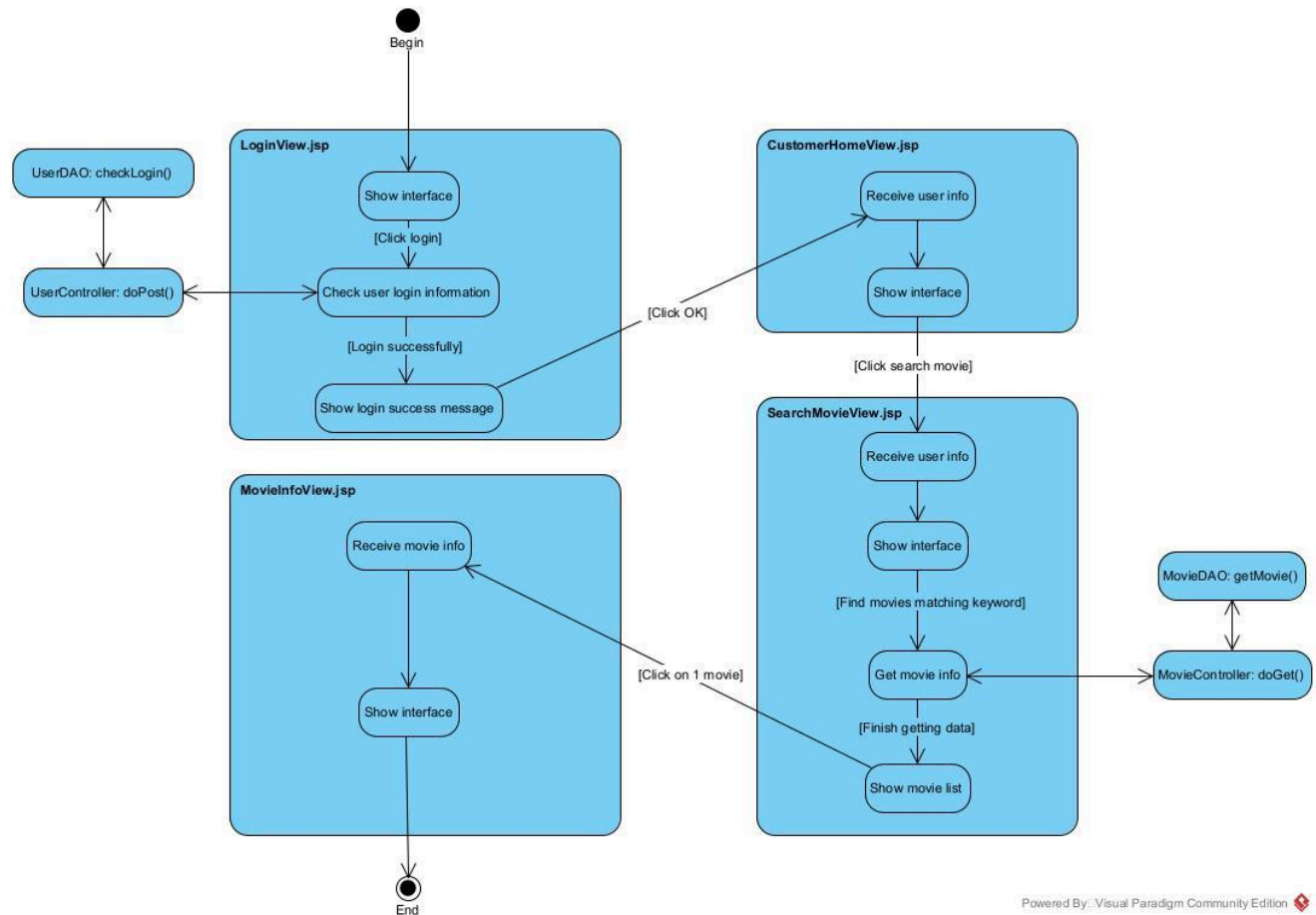
- Interface layer has the following jsp pages: LoginView.jsp, ManagementStaffHomeView.jsp, StatsView.jsp, CustomerStatsByRevenueView.jsp, TransactionView.jsp
- Controller layer has the following Controller classes: UserController, CustomerStatsController, InvoiceController
- Data access layer has the following DAO classes: UserDAO, CustomerStatsDAO, InvoiceDAO, DAO



3.2. Dynamic design

a. Customer search for movie information

Activity diagram:

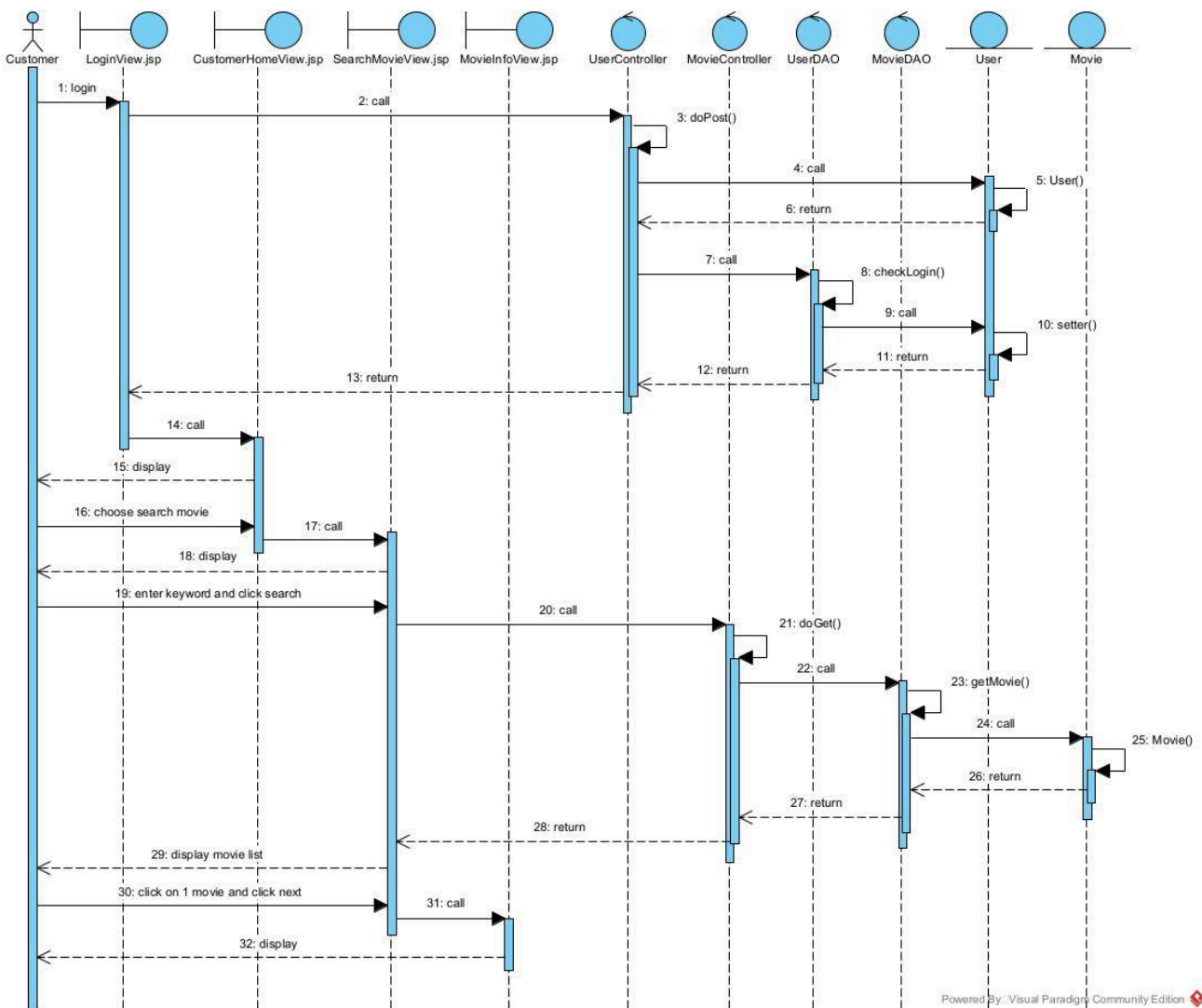


Scenario version 3:

1. The customer clicks the “Login” button on the interface after entering username and password.
2. The page LoginView.jsp calls the class UserController.
3. The class UserController calls the method doPost().
4. The method doPost() calls the class User to create a User object.
5. The class User packs the login information into a User object.
6. The class User returns a User object to the method doPost().
7. The method doPost() calls the class UserDAO to check login information.
8. The class UserDAO calls the method checkLogin().
9. The method checkLogin() calls the User class to pack the rest of the attributes.
10. The class User calls its set methods to set the rest of the attributes.
11. The class User returns the User object to the method checkLogin().
12. The method checkLogin() returns the result to the method doPost().
13. The method doPost() returns the result to the page LoginView.jsp
14. The page LoginView.jsp calls the page CustomerHomeView.jsp.
15. The page CustomerHomeView.jsp displays to the customer.

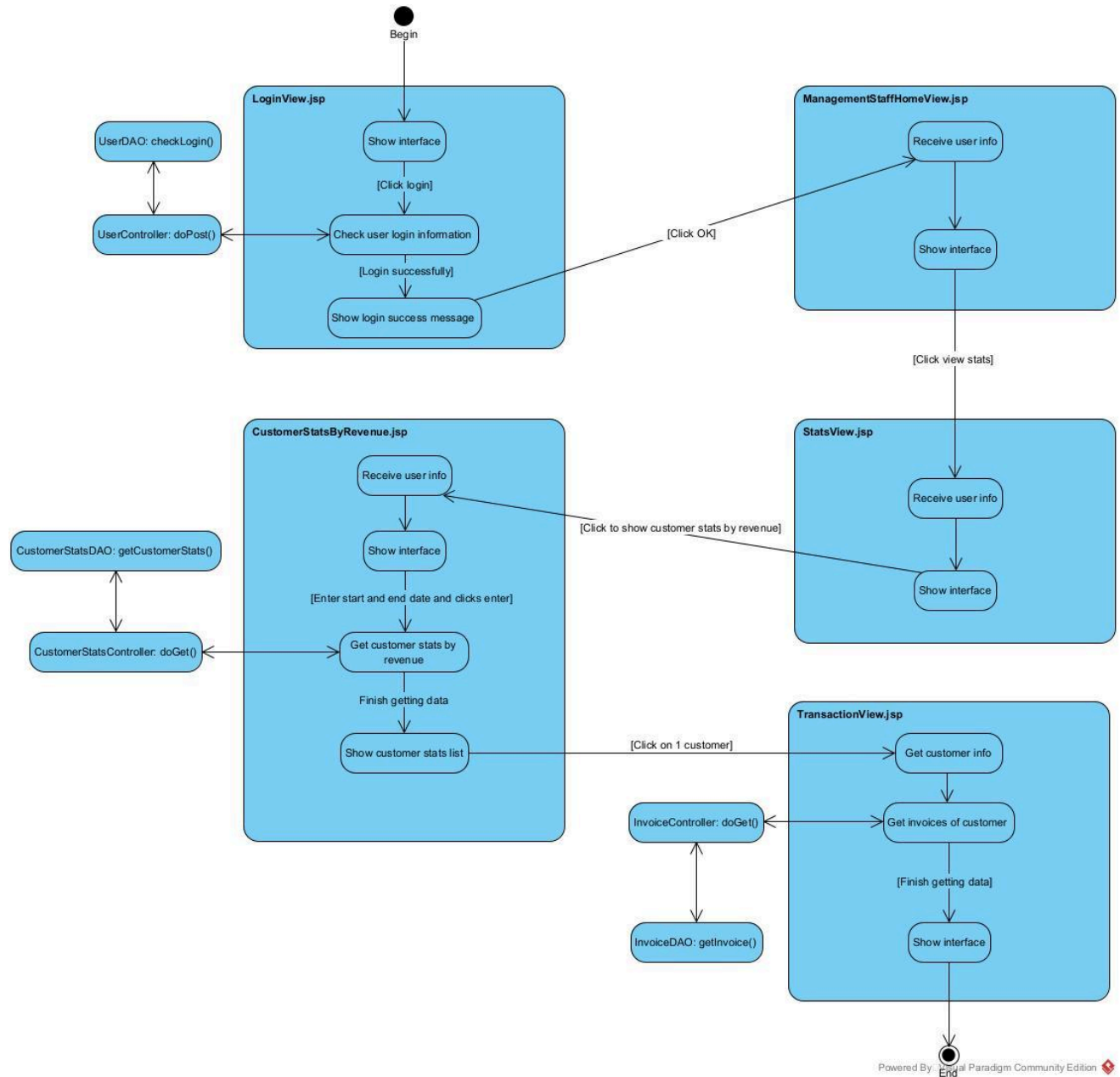
16. The customer clicks on the option to search for movies.
17. The page CustomerHomeView.jsp calls the page SearchMovieView.jsp.
18. The page SearchMovieView.jsp displays to the customer.
19. The customer enters the search keyword and clicks “Search”.
20. The page SearchMovieView.jsp calls the class MovieController.
21. The class MovieController calls the method doGet().
22. The method doGet() calls the class MovieDAO to find all movies whose name matches the keyword.
23. The class MovieDAO calls the method getMovie().
24. The method getMovie() calls the class Movie to pack the results.
25. The class Movie packs its attributes.
26. The class Movie returns the packed Movie object to the method getMovie().
27. The method getMovie() returns the results to the method doGet().
28. The method doGet() returns the results to the page SearchMovieView.jsp
29. The page SearchMovieView.jsp displays the results to the customer.
30. The customer chooses a movie and then clicks “Next”.
31. The page SearchMovieView.jsp calls the page MovieInfoView.jsp.
32. The page MovieInfoView.jsp displays to the customer.

Sequence diagram:



b. Management staff views customer statistics

Activity diagram:

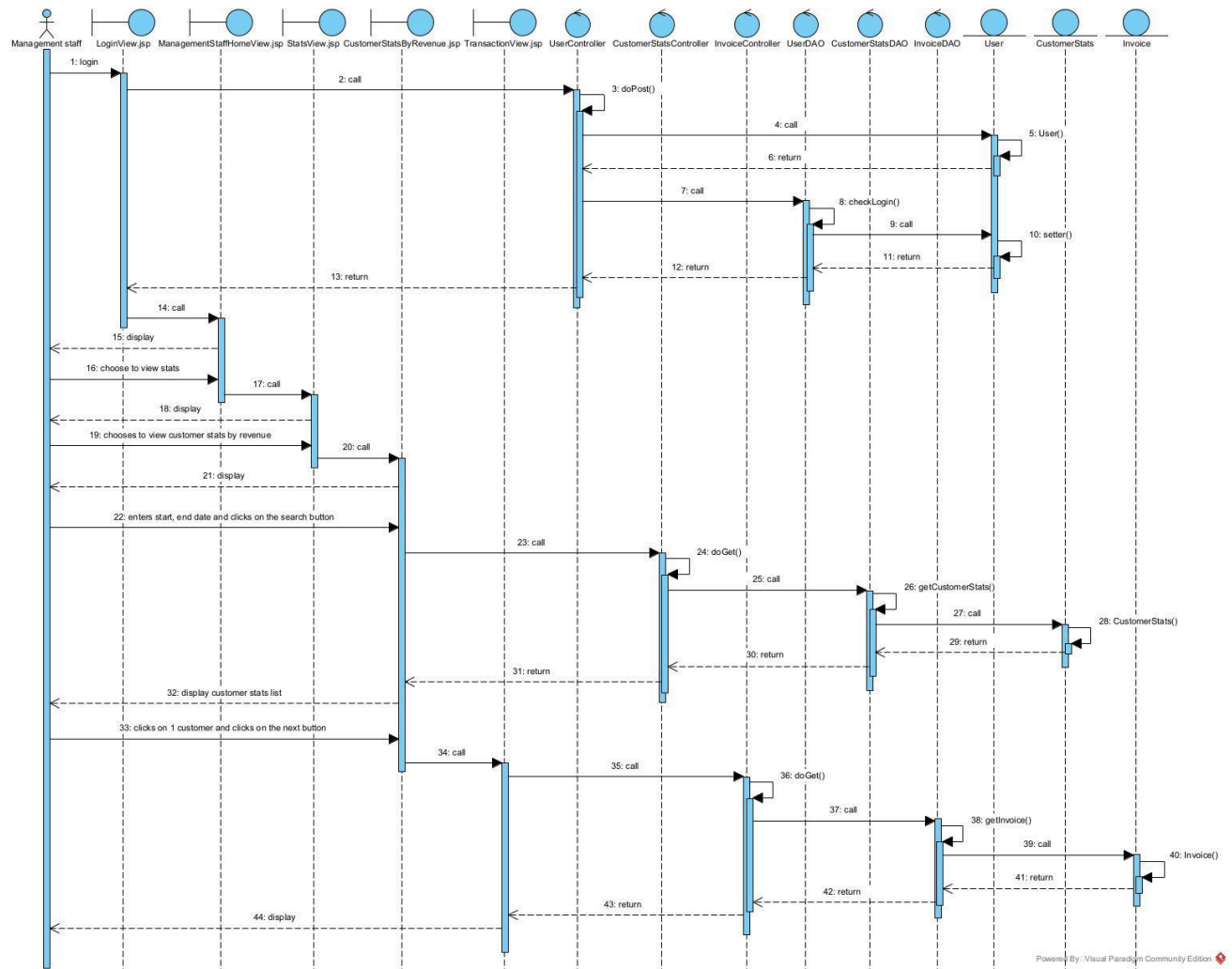


Scenario version 3:

1. The management staff clicks the “Login” button on the interface after entering username and password.
2. The page LoginView.jsp calls the class UserController.
3. The class UserController calls the method doPost().
4. The method doPost() calls the class User to create a User object.
5. The class User packs the login information into a User object.
6. The class User returns a User object to the method doPost().
7. The method doPost() calls the class UserDAO to check login information.
8. The class UserDAO calls the method checkLogin().
9. The method checkLogin() calls the User class to pack the rest of the attributes.

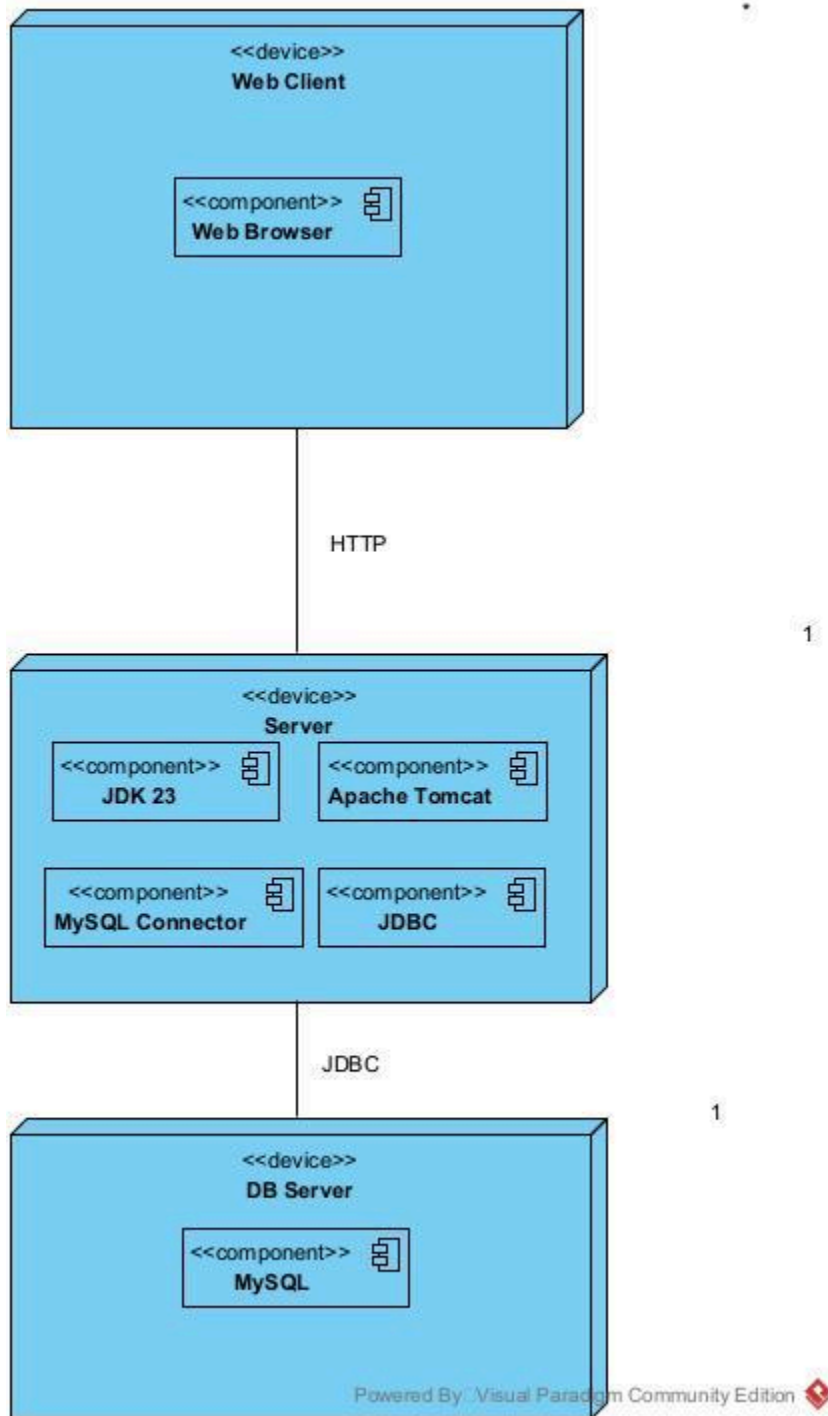
10. The class User calls its set methods to set the rest of the attributes.
11. The class User returns the User object to the method checkLogin().
12. The method checkLogin() returns the result to the method doPost().
13. The method doPost() returns the result to the page LoginView.jsp
14. The page LoginView.jsp calls the page ManagementStaffHomeView.jsp.
15. The page ManagementStaffHomeView.jsp displays to the management staff.
16. The management staff clicks on the option to view stats.
17. The page ManagementStaffHomeView.jsp calls the page ViewStats.jsp.
18. The page ViewStats.jsp displays to the management staff.
19. The management staff clicks on the option to view customer stats.
20. The page ViewStats.jsp calls the page CustomerStatsByRevenue.jsp.
21. The page CustomerStatsByRevenue.jsp displays to the management staff.
22. The management staff enters the start and end date and clicks "Search".
23. The page CustomerStatsByRevenue.jsp calls the class CustomerStatsController.
24. The class CustomerStatsController calls the method doGet().
25. The method doGet() calls the class CustomerStatsDAO to get a list of customer stats.
26. The class CustomerStatsDAO calls the method getCustomerStats().
27. The method getCustomerStats() calls the class CustomerStats to pack its attributes.
28. The class CustomerStats packs its attributes.
29. The class CustomerStats returns the packed results to the method getCustomerStats().
30. The method getCustomerStats() returns the result to the method doGet().
31. The method doGet() returns the result to the page CustomerStatsByRevenue.jsp.
32. The page CustomerStatsByRevenue.jsp displays the result to the management staff.
33. The management staff chooses a customer and clicks "Next".
34. The page CustomerStatsByRevenue.jsp calls the page TransactionView.jsp.
35. The page TransactionView.jsp calls the class InvoiceController.
36. The class InvoiceController calls the method doGet().
37. The method doGet() calls the class InvoiceDAO to get the list of all the invoices of the customer within the specified time period.
38. The class InvoiceDAO calls the method getInvoice().
39. The method getInvoice() calls the class Invoice to pack its attributes.
40. The class Invoice packs its attributes.
41. The class Invoice returns the packed Invoice object to the method getInvoice().
42. The method getInvoice() returns the result to the method doGet().
43. The method doGet() returns the result to the page TransactionView.jsp.
44. The page TransactionView.jsp displays to the management staff.

Sequence diagram:

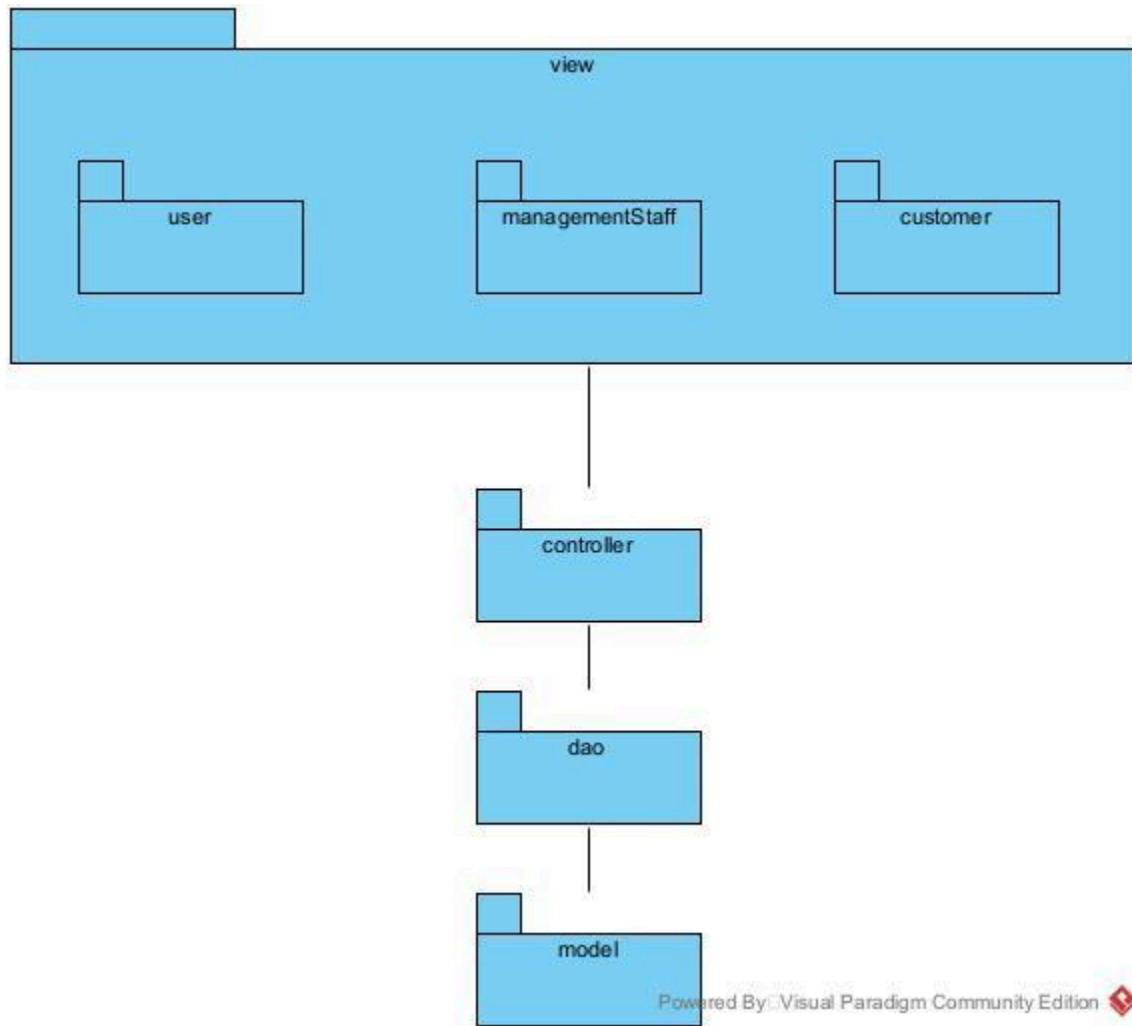


3.3. Deployment design

a. Deployment diagram



b. Package diagram



D. Coding

1. UI

1.1 LoginView.jsp

Cinema System

Login

Username:

Password:

Login

1.2 Customer search for movie information

CustomerHomeView.jsp

Cinema System

Customer Home

Nguyen Van A

Search movie

Logout

Cinema System

Movie Search

Please enter the
keyword:

Mưa

Search

Movie name	Release year
Mưa Đỏ	2025
Em Gái Mưa	2018
Con Mưa Ngang Qua	2018
Hát Dưới Mưa	1952

Back

Next

Cinema System

Movie Information

Name: Mưa Đỏ
Release date: 22/08/2025
Genre: Action
Length: 124 minutes
Description:

A feature movie on the theme of the revolutionary war, scripted by writer Chu Lai, inspired by and fictionalized from the 81 days and nights of heroic, resilient fighting by the people, cadres, and soldiers defending Quảng Trị Citadel in 1972.

[Back](#)[Home](#)

1.3. Management staff views customer statistics

ManagementStaffHomeView.jsp

Cinema System

Management Staff Home

Nguyen Xuan Kien

[View Statistics](#)[Logout](#)

Cinema System

Statistic

Customer Statistics

Back

CustomerStatsByRevenueView.jsp

Cinema System

Customer Statistics by Revenue

From: 08/06/2025 To: 18/06/2025 View

Customer Code	Customer name	Tickets purchased	Total revenue (in VND)
11	Pham Minh Hoang	5	1140000
4	Nguyen Van X	3	420000
5	Tran Thanh Y	3	420000
3	Dang Thu C	2	221000
7	Nguyen Thi B	2	210000
2	Nguyen Van A	2	180000
6	Nguyen Xuan A	0	0
12	Vu Ngoc Anh	0	0

Total tickets purchased: 17

Total revenue: 2591000 VND

Back

Next

Cinema System

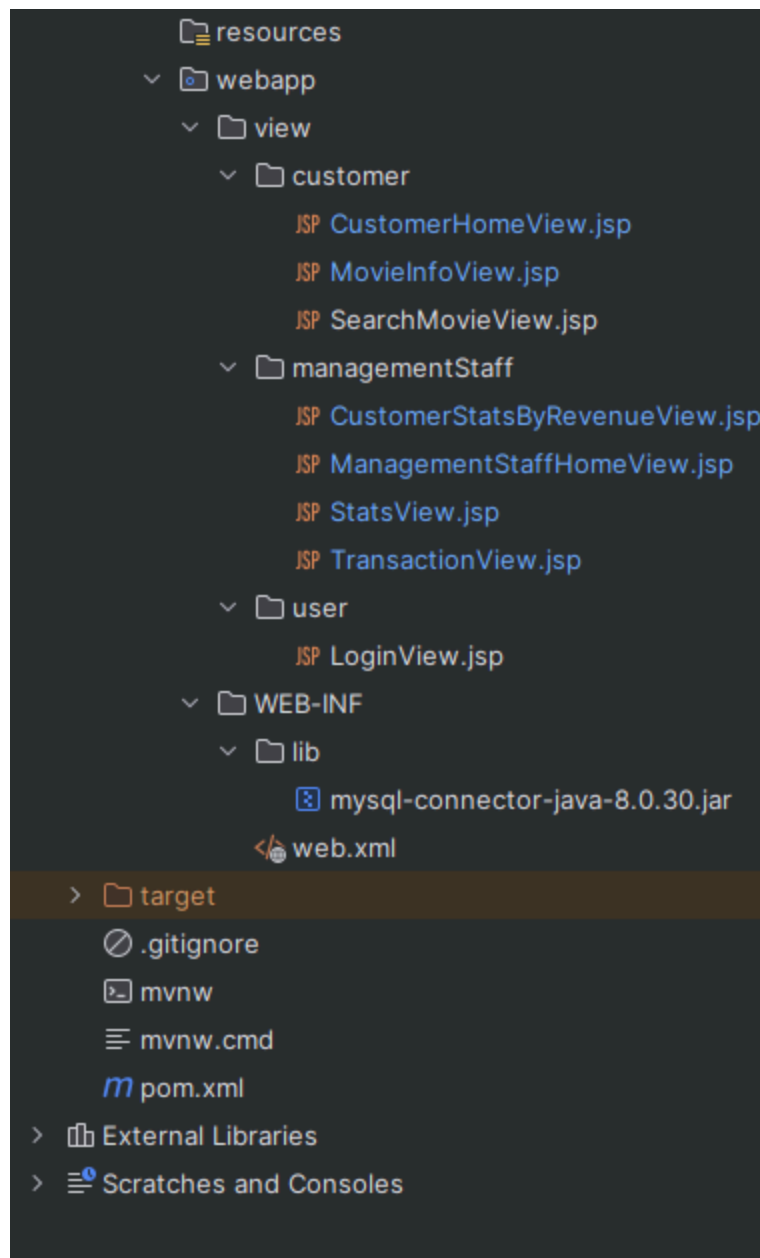
Customer Transactions

Customer code: 11
Customer name: Pham Minh Hoang
Customer date of birth: 2001-01-20
Customer phone: 0912222222
Customer email: hoangpm@example.com
Customer address: 15 Le Loi, Hai Phong

ID	Movie	Cinema	Theater	Seats	Watch time	Total price (in VND)	Saleoff	Method of order	Creation time	Saler
4	The Dark Knight	Galaxy Cinema	Theater 1	B5, B6	2025-06-15 20:15:00	300000	0%	counter	2025-06-15 20:00:00	Phung Canh Mo
7	Mưa Đỏ	Lotte Cinema Da Nang	Theater 4	E1, E2, E3	2025-06-19 13:30:00	840000	0%	online	2025-06-18 19:00:00	Phung Canh Mo

2. Folder structure

- ▼ cineman_pttk C:\Users\HLC_2021\IdeaProjects\cinema
- > .idea
- > .mvn
- ▼ src
 - ▼ main
 - ▼ java
 - ▼ controller
 - © CustomerStatsController
 - © InvoiceController
 - © MovieController
 - © UserController
 - ▼ dao
 - © CustomerStatsDAO
 - © DAO
 - © InvoiceDAO
 - © MovieDAO
 - © UserDAO
 - ▼ model
 - © CardType
 - © Cinema
 - © Customer
 - © CustomerStats
 - © Invoice
 - © ManagementStaff
 - © MembershipCard
 - © Movie
 - © MovieStats
 - © Saler
 - © ScreenType
 - © Seat
 - © SeatType
 - © Showtime
 - © Staff
 - © Theater
 - © Ticket
 - © User
 - resources
 - ▼ webapp
 - ▼ view
 - ▼ customer
 - JSP CustomerHomeView.jsp
 - JSP MovieInfoView.jsp
 - JSP SearchMovieView.jsp
 - ▼ managementStaff
 - JSP CustomerStatsByRevenueView.jsp
 - JSP ManagementStaffHomeView.jsp
 - JSP StatsView.jsp
 - JSP TransactionView.jsp



3. Project link

https://github.com/r0fing/cineman_pttk